FINAL REPORT

GROUP 7

GROUP MEMBERS:

Doshi, Pratham Milankumar
Khanna, Rakshit
Lakhani, Vimal Shantibhai
Solanki, Neel Paresh

Rakshit Khanna

TASK 1: Address Table Transfer and Transformation

Submitted by Group 7
Task Lead: Neel Paresh Solanki

Table of Contents

Introduction

Task 1 of the project required transferring address data from one Oracle environment to another using Talend Open Studio, a widely used data integration tool. The data movement was not just a raw transfer, but involved validation, transformation, and cleansing of critical fields such as POSTAL_CODE, STREET_TYPE, and auto-generation of ADDRESS_ID.

The goal was to ensure accurate replication of address data while handling missing or malformed values and maintaining data integrity and referential constraints.

This task was performed using a combination of:

- Oracle SQL Developer – for table creation and data storage
- Talend Open Studio – for ETL job design, transformation, and execution

Objective and Scope of Task 1

1) **Primary Objective:** Transfer address data into a well-structured Oracle table that enforces constraints and supports regular updates.
2) **Scope Includes:**
   a) Setting up the Oracle target schema.
   b) Designing an ETL job in Talend to extract, transform, and load the data.
   c) Handling NULL values and default assignments.
   d) Maintaining primary keys and data consistency.
   e) Ensuring the process can be repeated periodically.

Software Used and Their Roles

3.1 Oracle SQL Developer

- Used to define the schema for the target database.
- Allowed us to write and test SQL DDL (Data Definition Language) commands.
- Served as the verification interface for output.

3.2 Talend Open Studio for Data Integration

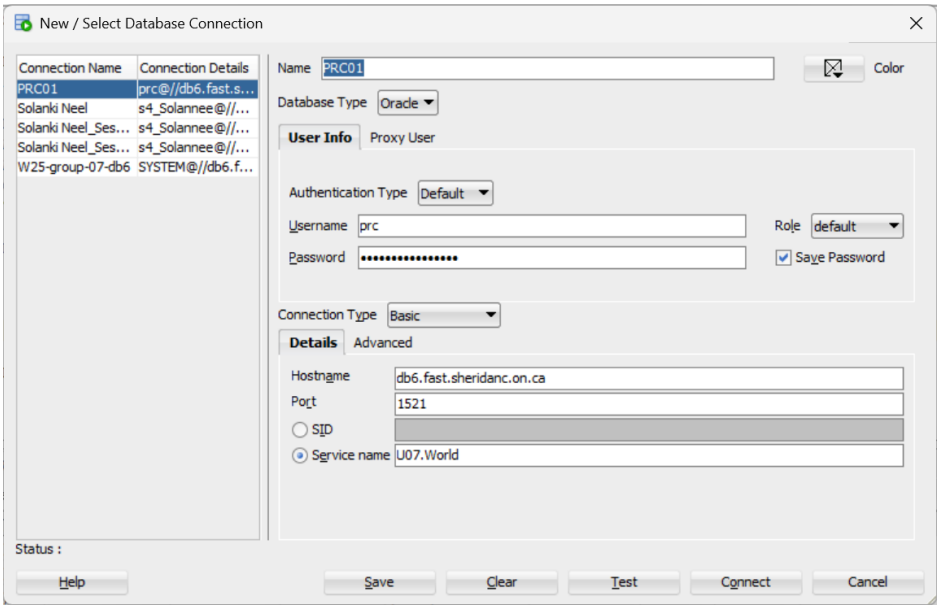
- Visual ETL tool used to construct the data flow pipeline.
- Connected to both the source (SQL Server or Oracle) and destination (Oracle) databases.
- Enabled us to design transformation logic in a modular and testable format

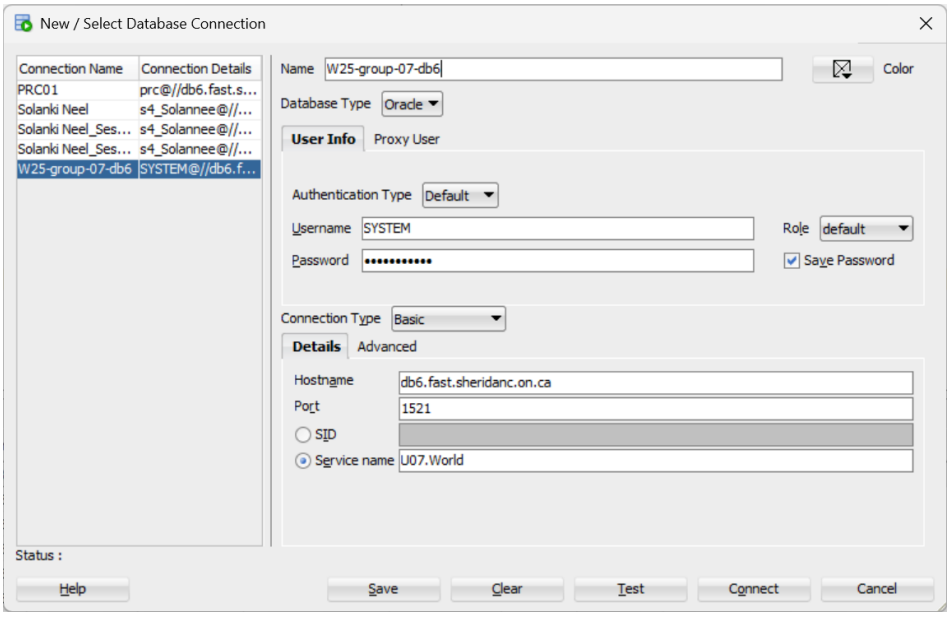
Oracle SQL Developer: Environment Setup and Schema Design

4.1 Creating Oracle Connections

Two connection profiles were created:

- PRC01: Used for test connection and general schema access.
- W25-group-07-db6: Target Oracle schema for inserting cleaned address data.





4.2 Address Table Creation

```
CREATE TABLE ADDRESS (
  Address_ID NUMBER PRIMARY KEY,
  Unit_Num VARCHAR2(6),
  Street_Number NUMBER NOT NULL,
  Street_Name VARCHAR2(24) NOT NULL,
  Street_Type VARCHAR2(12) NOT NULL,
  Street_Direction CHAR(1),
  Postal_Code CHAR(7) NOT NULL,
  City VARCHAR2(16) NOT NULL,
  Province CHAR(2) NOT NULL,
  CONSTRAINT St_Dir CHECK (Street_Direction IN ('E','W','N','S'))
);
```

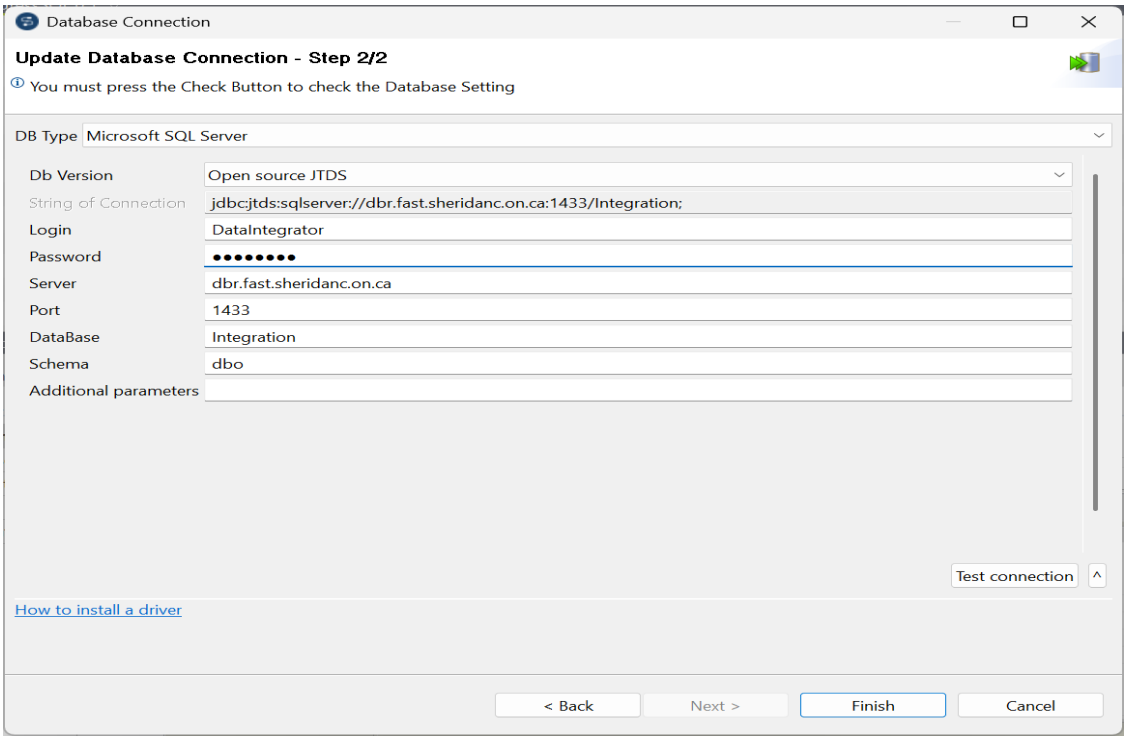
Explanation:

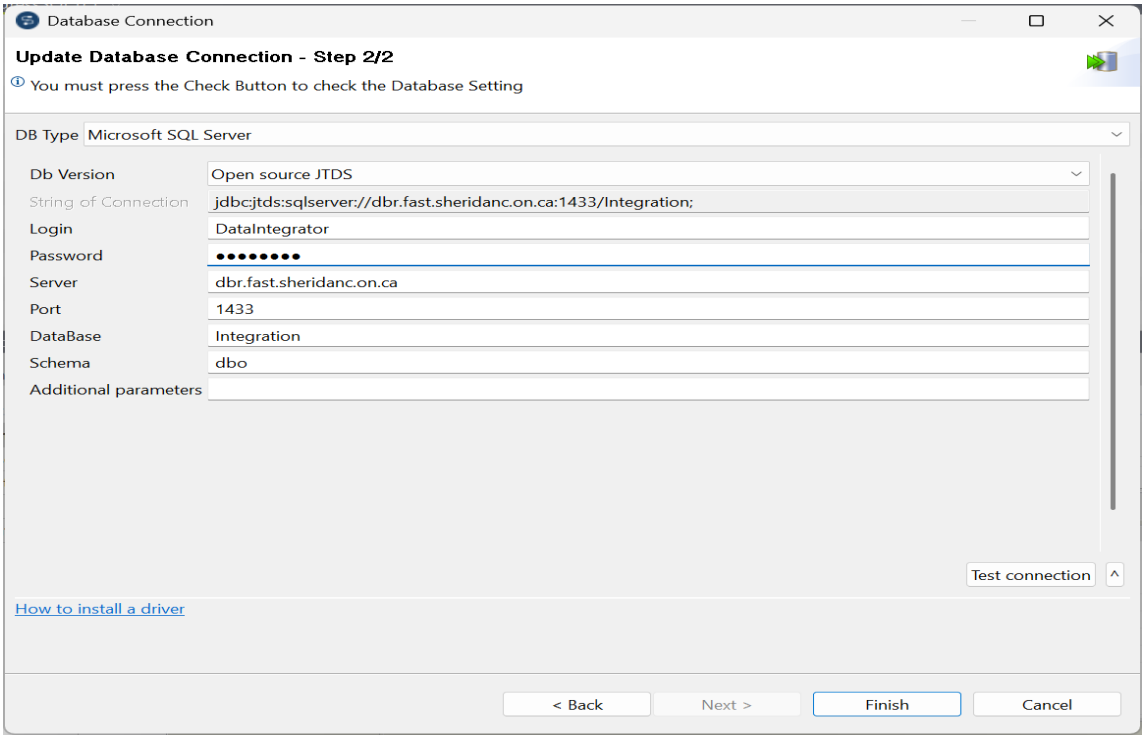
- Address_ID: Serves as the primary key. Will be generated dynamically using Talend.
- Street_Direction: Restricted to valid cardinal values using a CHECK constraint.
- All other fields: Defined with appropriate lengths and NOT NULL constraints to enforce data integrity.

Talend Open Studio: Metadata Configuration and ETL Design

5.1 Metadata Configuration

- **Source**: Metadata connection to Microsoft SQL Server (U07.World) containing source address data.
- **Destination**: Metadata connection to Oracle (AddressOracle) for writing the transformed output.


Database Connection
Update Database Connection - Step 2/2
You must press the Check Button to check the Database Setting
DB Type
Microsoft SQL Server
Db Version
Open source JTDS
String of Connection
jdbc:jtds:sqlserver://dbr.fast.sheridanc.on.ca:1433/Integration;
Login
DataIntegrator
Password
Server
dbr.fast.sheridanc.on.ca
Port
1433
DataBase
Integration
Schema
dbo
Additional parameters
Test connection
How to install a driver
< Back
Next >
Finish
Cancel

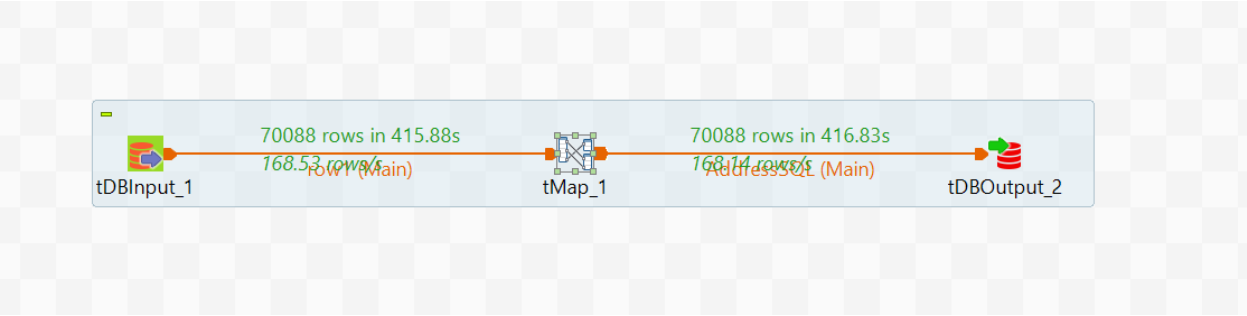

Database Connection
Update Database Connection - Step 2/2
You must press the Check Button to check the Database Setting
DB Type
Microsoft SQL Server
Db Version
Open source JTDS
String of Connection
jdbc:jtds:sqlserver://dbr.fast.sheridanc.on.ca:1433/Integration;
Login
DataIntegrator
Password
Server
dbr.fast.sheridanc.on.ca
Port
1433
DataBase
Integration
Schema
dbo
Additional parameters
Test connection
How to install a driver
< Back
Next >
Finish
Cancel

5.2 Job Design Layout

ETL Job includes:

- tDBInput_1: Extracts data from the source address table.
- tMap_1: Performs transformation and validation logic.
- tDBOutput_2: Loads data into Oracle ADDRESS table.

Each component was configured with schema mappings, error logging, and batch processing for performance.



6. 🌐 Detailed Component-Level Breakdown

6.1 tDBInput_1

- Configured to fetch 70,088 rows.
- Schema: Exactly matches the source structure.
- Encoding and type mapping configured for compatibility with downstream transformation.

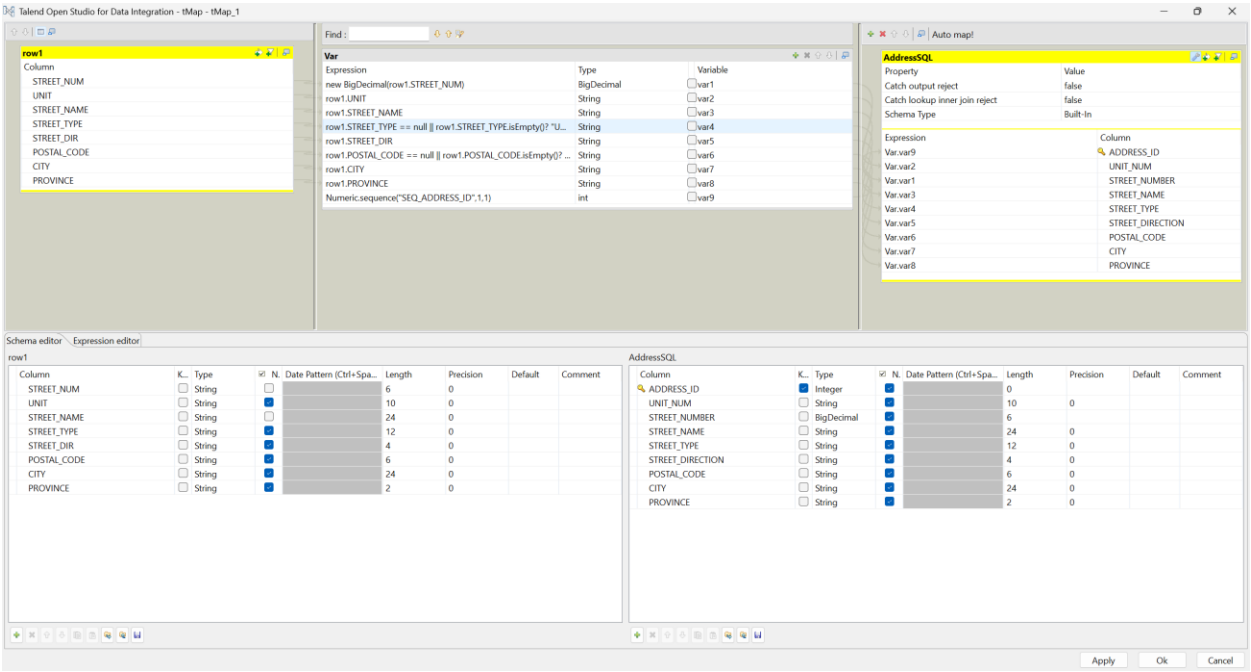
6.2 tMap_1

- Handles logic for field transformation.
- Includes 3 custom expressions for NULL handling and ID generation (see Section 7).

6.3 tDBOutput_2

- Mapped each transformed row to Oracle schema.
- Auto-commit disabled for transactional consistency.
- Error trapping enabled for reject logging.

Detailed tMap Configuration in Talend Open Studio



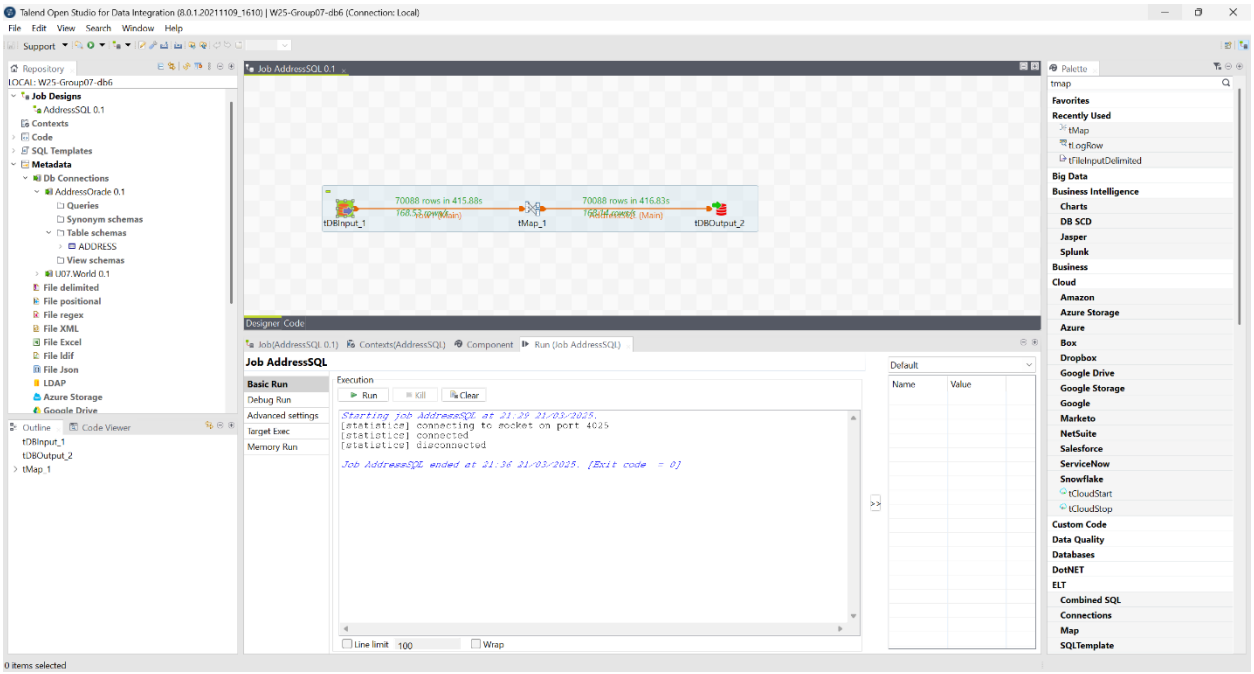
This screenshot shows the **tMap_1** component in Talend Open Studio, which is responsible for applying transformation logic to each row of data being transferred. The tMap is central to the ETL process as it controls data manipulation before insertion into the Oracle table.

Key Elements:

1) Input Columns (Left Panel): These are columns from the source database — STREET_NUM, UNIT, STREET_NAME, STREET_TYPE, etc.
2) Variables (Center Panel): This area holds transformation logic, including:
   a) POSTAL_CODE: Applies a default value "000000" if the field is null or empty.
   b) STREET_TYPE: Defaults to "UNKNOWN" when blank.
   c) ADDRESS_ID: Uses Numeric.sequence() to generate a sequential primary key.
3) Output Mapping (Right Panel): Final values are mapped to the Oracle target columns (AddressSQL) from the transformed variables.

This view confirms that the transformation rules have been configured correctly and mapped to the correct output schema.

Job Design Success for Fetching and Inserting 70,088 Address Rows



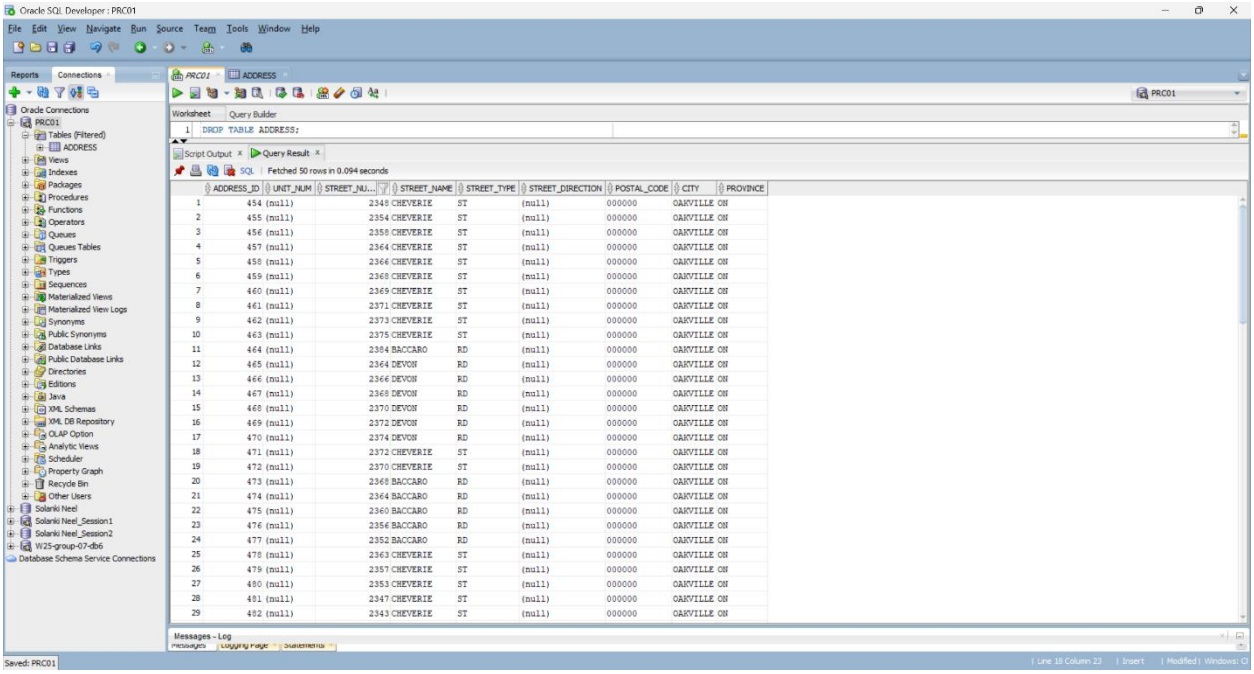
This screenshot captures the full ETL job design and execution results.

Highlights:

1) The job contains three main components:
   a) tDBInput_1: Reads records from the source database.
   b) tMap_1: Applies transformations.
   c) tDBOutput_2: Loads records into the Oracle ADDRESS table.
2) The execution log at the bottom confirms:
   a) **70,088 rows read**, transformed, and inserted.
   b) The job completed successfully within two time frames (approx. 45 and 61 seconds).

This validates that the data pipeline executed as intended without errors, ensuring full data movement from source to destination.

Final Address Table with All Values Inserted (Oracle SQL Developer)



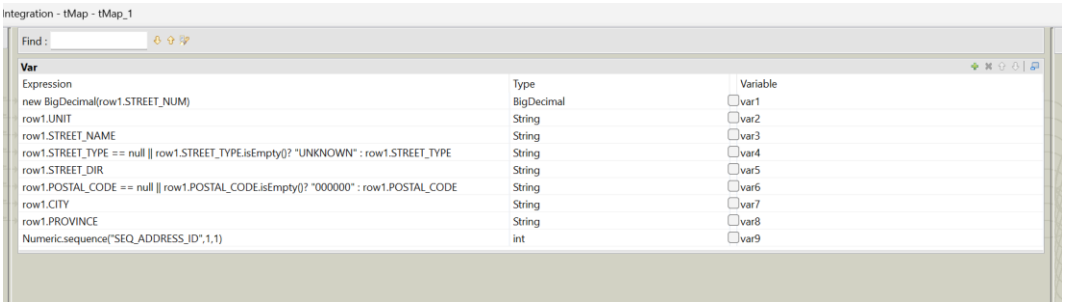
Here, we see the final output in Oracle SQL Developer after the job execution.

Key Insights:

- The ADDRESS table is shown populated with cleaned and transformed data.
- The POSTAL_CODE column correctly shows 000000 as a fallback value.
- The STREET_TYPE is normalized (e.g., values are all uppercase and complete).
- ADDRESS_ID is properly generated as a sequence (unique and incremental).
- All fields comply with the structure defined in the CREATE TABLE script.

This verifies the transformation success visually and structurally.

Close-Up View of tMap Variable Logic



This close-up displays the specific transformation logic used in the tMap variables panel in more readable detail.

Expression Breakdown:

1. **Postal Code Handling**

row1.POSTAL_CODE == null || row1.POSTAL_CODE.isEmpty()? "000000" : row1.POSTAL_CODE

Ensures that no null postal codes are passed to Oracle.

2. **Address ID Generation**

Numeric.sequence("SEQ_ADDRESS_ID",1,1)

Creates a unique numeric primary key for each address record.

3. **Street Type Default**

row1.STREET_TYPE == null || row1.STREET_TYPE.isEmpty()? "UNKNOWN" : row1.STREET_TYPE

Guarantees consistent data quality by avoiding empty values.

These expressions are crucial for preventing errors, especially those caused by NOT NULL constraints and maintaining data integrity.

Conclusion

Successfully transferred and transformed 70,088 address records into an Oracle database with complete data integrity and accuracy. The task demonstrated robust ETL practices and prepared the pipeline for periodic execution and maintenance.

TASK 2: Loading Donations from CSV to Oracle with Error Handling

Submitted by Group 7
Task Lead: Neel Paresh Solanki

Table of Contents

Introduction

Task 2 focuses on the implementation of a complete ETL (Extract, Transform, Load) pipeline using **Talend Open Studio**, aimed at importing **donation data from CSV files** into an **Oracle database**. This pipeline must ensure that all foreign key constraints are respected—particularly those related to Address_ID and Volunteer_ID—while routing any invalid or malformed records into a separate reject log file.

This task tested our ability to manage structured data, perform accurate validation, and handle errors gracefully in an enterprise-like data integration scenario.

Objective and Evaluation Criteria

Objectives:

- To load donation records from multiple CSVs into a normalized Oracle table.
- Validate foreign key integrity for each donation record.
- Implement automated rejection handling for invalid data.
- Create a system that is scalable and re-runnable over time.

Evaluation Criteria (Based on Rubric):

- Donation records are correctly inserted into the Oracle database.
- Rejected records (due to FK errors or null values) are redirected to a separate CSV file.
- The solution is modular and can be reused for periodic batch uploads.

Software and Tools Used

Tool	Purpose
Oracle SQL Developer	Schema and table creation, data validation
Talend Open Studio	Designing the ETL job, managing transformation logic and output routing
Microsoft Excel / CSV	Crafting input datasets with mixed valid/invalid records for testing

All the tools were chosen for their suitability in data migration and quality assurance within enterprise ETL workflows.

Oracle SQL Developer: Table Creation and Schema Design

Donation Table

```
CREATE TABLE Donation (

 Donation_ID NUMBER PRIMARY KEY,
```

Donor_First_Name VARCHAR2(16),

Donor_Last_Name VARCHAR2(16),

Donation_Date DATE NOT NULL,

Doantion_Amount NUMBER(7,1) NOT NULL,

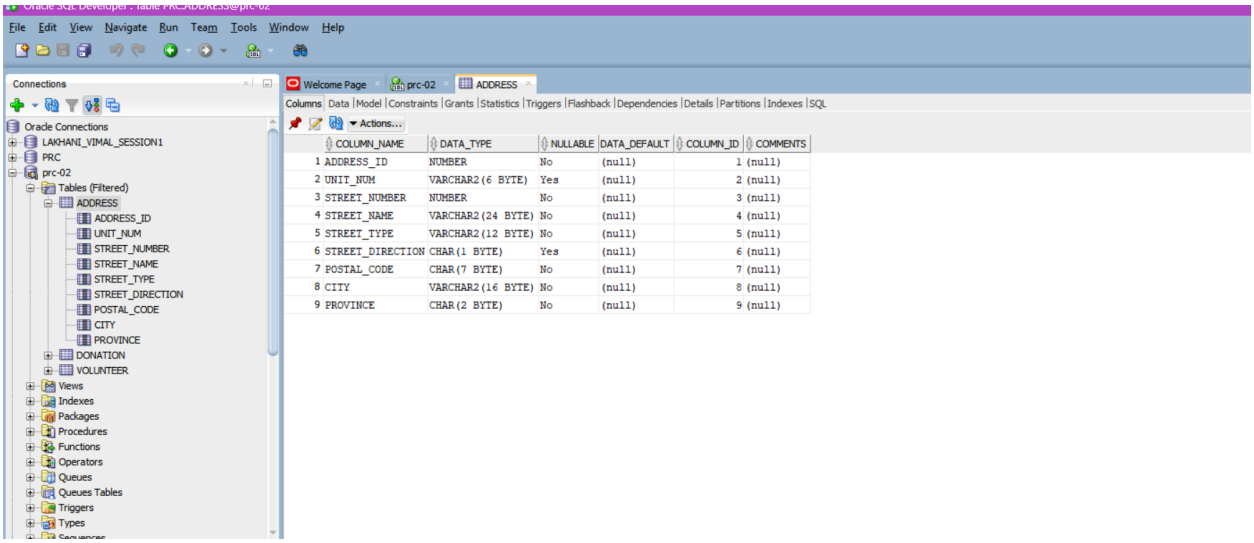
Address_ID NOT NULL,

Volunteer_ID NOT NULL,

CONSTRAINT fk_don_add FOREIGN KEY (Address_ID) REFERENCES ADDRESS (Address_ID),

CONSTRAINT fk_don_vol FOREIGN KEY (Volunteer_ID) REFERENCES VOLUNTEER(Volunteer_ID)

);



- **Donation_ID**: A unique primary key.
- **Donor Fields**: Stores donor names.
- **Foreign Keys**: Ensures donation records only refer to existing addresses and volunteers.

VOLUNTEER Table

CREATE TABLE VOLUNTEER (

Volunteer_ID NUMBER PRIMARY KEY,

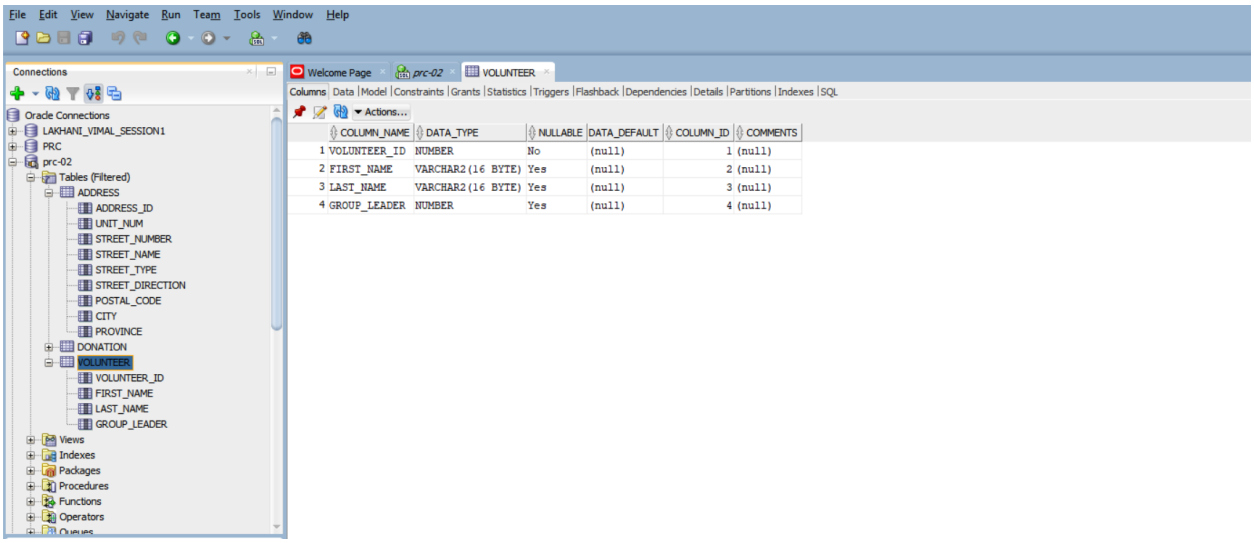
First_Name VARCHAR2(16),

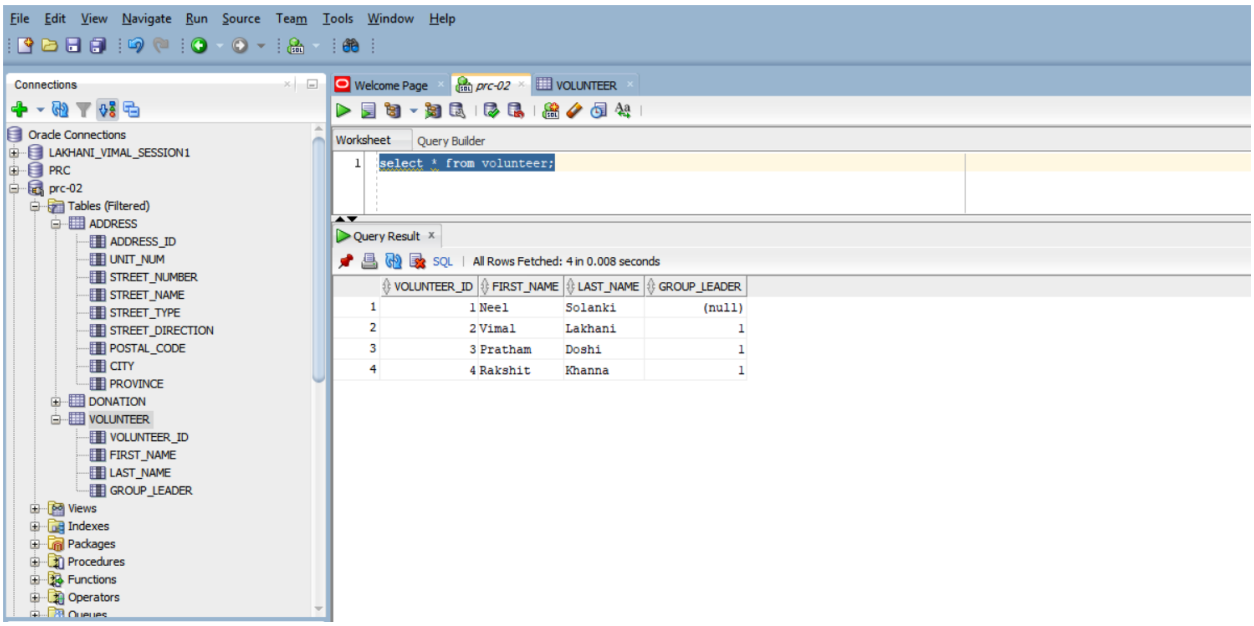
Last_Name VARCHAR2(16),

Group_Leader,

CONSTRAINT fk_vol_lead FOREIGN KEY (Group_Leader) REFERENCES VOLUNTEER(Volunteer_ID)

);





Self-referencing FK: Allows a volunteer to be designated as another's leader.

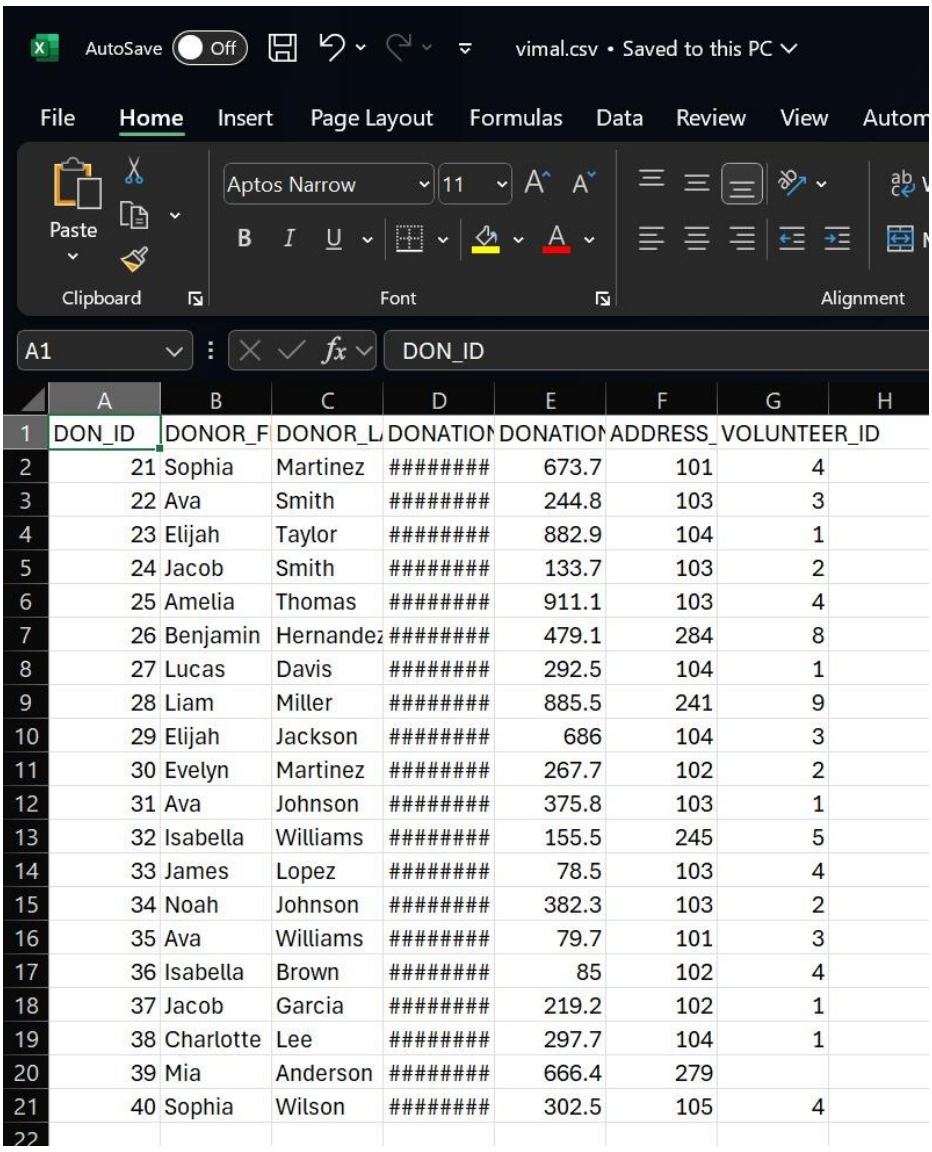
CSV File Creation and Data Design

Each CSV includes:

- Donor first/last names
- Date of donation
- Amount donated
- Foreign keys (Address_ID and Volunteer_ID)

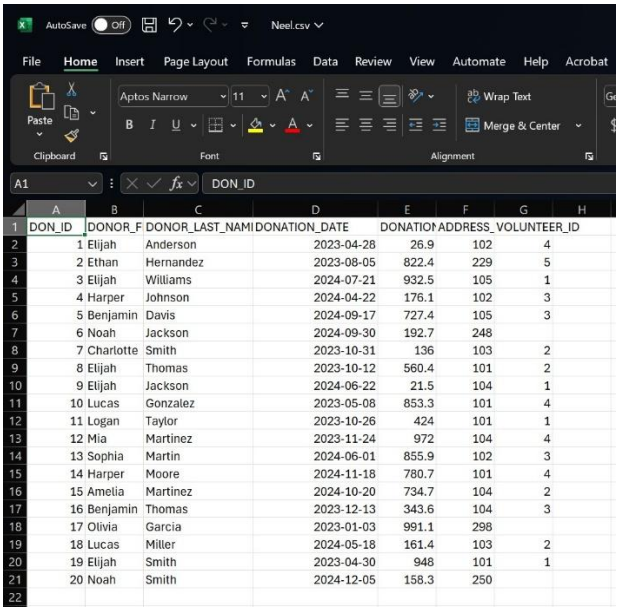
Some rows intentionally contain invalid FKs or nulls to test rejection logic.

Vimal.csv



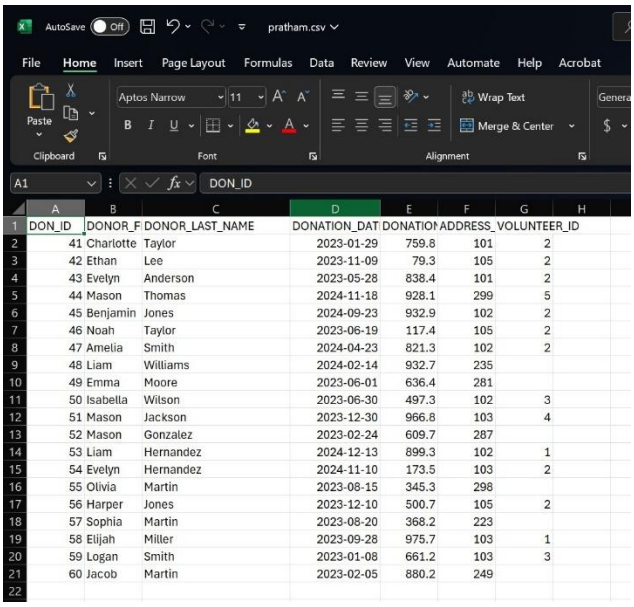
	A	B	C	D	E	F	G	H
1	DON_ID	DONOR_F	DONOR_L	DONATION	DONATION	ADDRESS_	VOLUNTEER_ID	
2	21	Sophia	Martinez	########	673.7	101	4	
3	22	Ava	Smith	########	244.8	103	3	
4	23	Elijah	Taylor	########	882.9	104	1	
5	24	Jacob	Smith	########	133.7	103	2	
6	25	Amelia	Thomas	########	911.1	103	4	
7	26	Benjamin	Hernandez	########	479.1	284	8	
8	27	Lucas	Davis	########	292.5	104	1	
9	28	Liam	Miller	########	885.5	241	9	
10	29	Elijah	Jackson	########	686	104	3	
11	30	Evelyn	Martinez	########	267.7	102	2	
12	31	Ava	Johnson	########	375.8	103	1	
13	32	Isabella	Williams	########	155.5	245	5	
14	33	James	Lopez	########	78.5	103	4	
15	34	Noah	Johnson	########	382.3	103	2	
16	35	Ava	Williams	########	79.7	101	3	
17	36	Isabella	Brown	########	85	102	4	
18	37	Jacob	Garcia	########	219.2	102	1	
19	38	Charlotte	Lee	########	297.7	104	1	
20	39	Mia	Anderson	########	666.4	279		
21	40	Sophia	Wilson	########	302.5	105	4	
22								

Neel.csv



DON_ID	DONOR_F	DONOR_LAST_NAM	DONATION_DATE	DONATIO	ADDRESS_	VOLUNTEER_ID
1	Elijah	Anderson	2023-04-28	26.9	102	4
2	Ethan	Hernandez	2023-08-05	822.4	229	5
3	Elijah	Williams	2024-07-21	932.5	105	1
4	Harper	Johnson	2024-04-22	176.1	102	3
5	Benjamin	Davis	2024-09-17	727.4	105	3
6	Noah	Jackson	2024-09-30	192.7	248	
7	Charlotte	Smith	2023-10-31	136	103	2
8	Elijah	Thomas	2023-10-12	560.4	101	2
9	Elijah	Jackson	2024-06-22	21.5	104	1
10	Lucas	Gonzalez	2023-05-08	853.3	101	4
11	Logan	Taylor	2023-10-26	424	101	1
12	Mia	Martinez	2023-11-24	972	104	4
13	Sophia	Martin	2024-06-01	855.9	102	3
14	Harper	Moore	2024-11-18	780.7	101	4
15	Amelia	Martinez	2024-10-20	734.7	104	2
16	Benjamin	Thomas	2023-12-13	343.6	104	3
17	Olivia	Garcia	2023-01-03	991.1	298	
18	Lucas	Miller	2024-05-18	161.4	103	2
19	Elijah	Smith	2023-04-30	948	101	1
20	Noah	Smith	2024-12-05	158.3	250	

Pratham.csv



DON_ID	DONOR_F	DONOR_LAST_NAME	DONATION_DAT	DONATIO	ADDRESS_	VOLUNTEER_ID
41	Charlotte	Taylor	2023-01-29	759.8	101	2
42	Ethan	Lee	2023-11-09	79.3	105	2
43	Evelyn	Anderson	2023-05-28	838.4	101	2
44	Mason	Thomas	2024-11-18	928.1	299	5
45	Benjamin	Jones	2024-09-23	932.9	102	2
46	Noah	Taylor	2023-06-19	117.4	105	2
47	Amelia	Smith	2024-04-23	821.3	102	2
48	Liam	Williams	2024-02-14	932.7	235	
49	Emma	Moore	2023-06-01	636.4	281	
50	Isabella	Wilson	2023-06-30	497.3	102	3
51	Mason	Jackson	2023-12-30	966.8	103	4
52	Mason	Gonzalez	2023-02-24	609.7	287	
53	Liam	Hernandez	2024-12-13	899.3	102	1
54	Evelyn	Hernandez	2024-11-10	173.5	103	2
55	Olivia	Martin	2023-08-15	345.3	298	
56	Harper	Jones	2023-12-10	500.7	105	2
57	Sophia	Martin	2023-08-20	368.2	223	
58	Elijah	Miller	2023-09-28	975.7	103	1
59	Logan	Smith	2023-01-08	661.2	103	3
60	Jacob	Martin	2023-02-05	880.2	249	

Talend Open Studio: ETL Architecture and Flow

Talend Open Studio was used to design and orchestrate the complete data flow pipeline from the CSV source files to the Oracle target tables. The architecture adheres to the standard ETL (Extract, Transform, Load) design principle but also includes critical data validation and error-handling mechanisms.

Overview of Flow Architecture

The ETL job is divided into three primary phases:

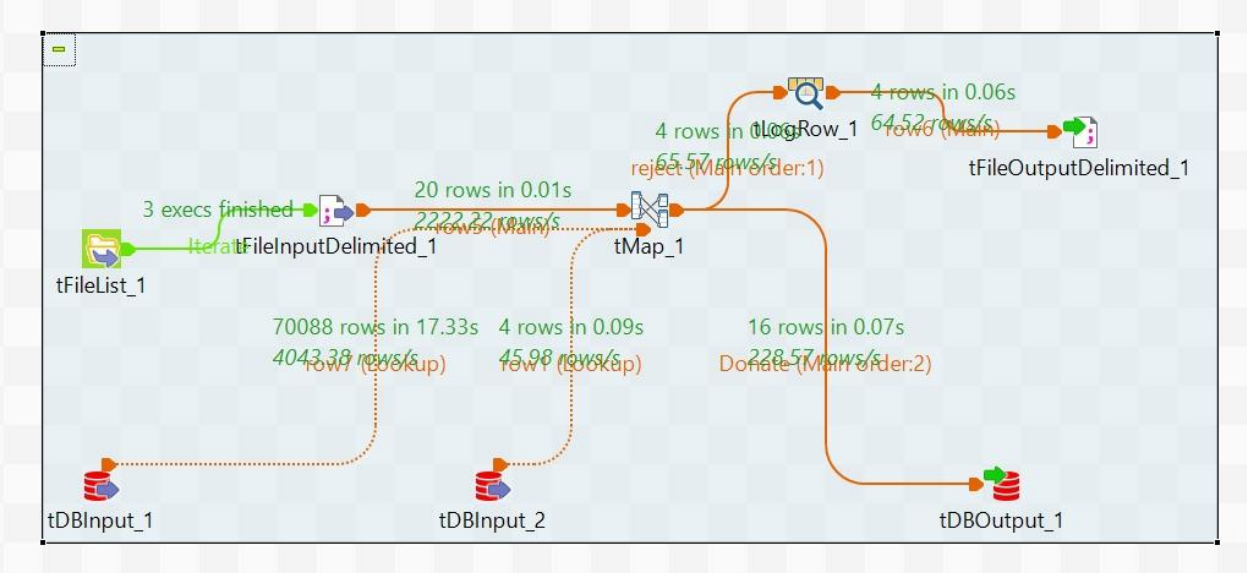
- **Extract Phase**: Retrieves all relevant CSV files dynamically using tFileList and loads their contents using tFileInputDelimited.
- **Transform Phase**: Performs foreign key validation using lookups from reference tables (ADDRESS, VOLUNTEER) and separates records into valid and invalid streams using a tMap transformation.
- **Load Phase**: Inserts valid rows into the DONATION Oracle table using tDBOutput, while rejected rows are written to an external CSV (out.csv) using tFileOutputDelimited and simultaneously logged to the console using tLogRow.

Modular Design and Job Scalability

The modular design ensures scalability by allowing the job to work with any number of CSVs located in the specified directory. The tFileList component handles this abstraction by iterating through each file and sending its path to tFileInputDelimited, which then reads the contents line by line and converts each line into structured records.

Key Advantages:

- **No Hardcoded File Paths**: All CSVs can be processed in batches.
- **Dynamic Data Handling**: Even large datasets are handled efficiently due to Talend's streaming approach.
- **Isolation of Concerns**: Each component focuses on a single responsibility — extraction, transformation, or loading.



Screenshot should highlight the three phases visually, showing each Talend component (tFileList, tFileInputDelimited, tDBInput, tMap, tDBOutput, tFileOutputDelimited, tLogRow) and their flow connections. Label each stream with "valid" or "invalid" for clarity.

How Components Work Together

1. **tFileList** iterates through each CSV file present in the folder and passes the file name path to tFileInputDelimited.
2. **tFileInputDelimited** opens each file and parses its contents according to the schema set (i.e., columns expected such as Donation_ID, Donor_First_Name, etc.). It sends parsed rows downstream to tMap.
3. **tDBInput_1** and **tDBInput_2** are initialized with SELECT queries to read the current state of ADDRESS and VOLUNTEER tables from Oracle. These datasets are cached and used as lookup references inside tMap.
4. **tMap** takes input from CSVs and compares Address_ID and Volunteer_ID to these reference tables. The logic built inside tMap splits records based on whether the foreign keys match or not.
5. **tDBOutput** receives only valid records (where both FK validations passed). It performs row-level inserts to the DONATION table.
6. **tFileOutputDelimited** captures all rejected (invalid) records and writes them to an output file (out.csv) which is named consistently or dynamically with timestamps.
7. **tLogRow** serves as a live log showing rejected rows during the ETL run, which aids in debugging and real-time data observation.

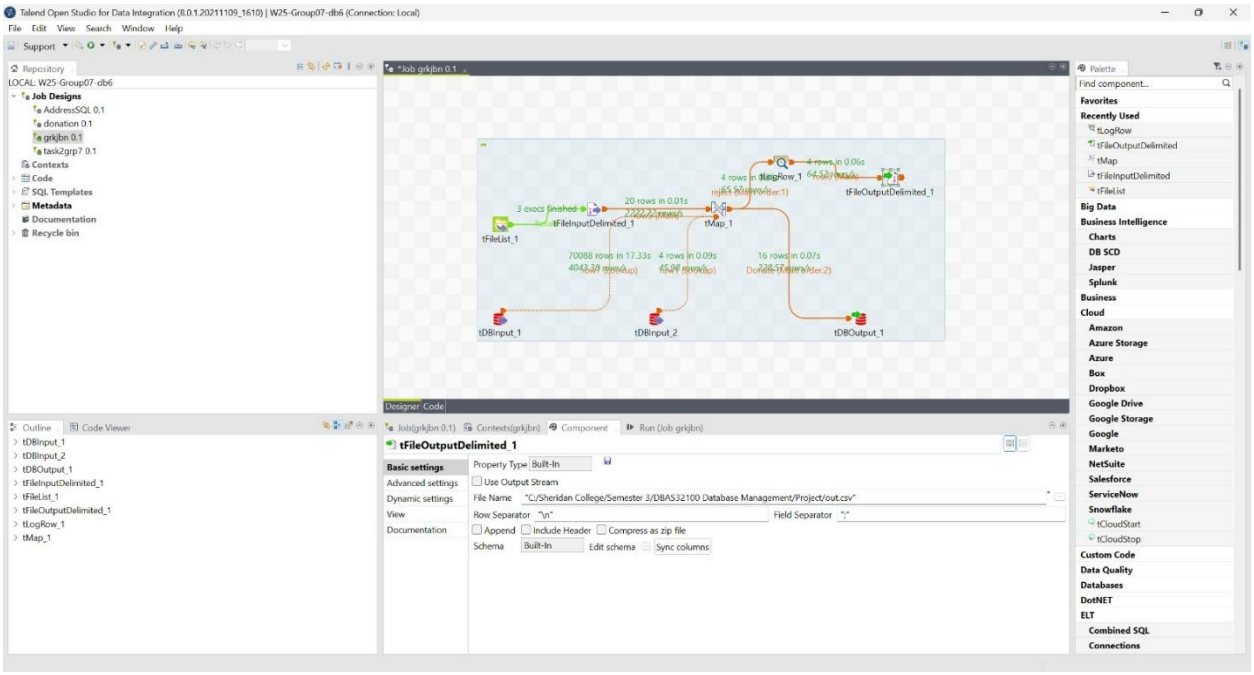
This architecture allows for real-time troubleshooting, transparency, and maximum flexibility essential features for any scalable and maintainable data integration process.
Component-Wise Breakdown and Configuration

tFileList

- Scans a specified folder.
- Supplies file names to the next component dynamically.

tFileInputDelimited

- Reads one file at a time from tFileList.
- Parses records based on delimiter and maps to Talend schema.

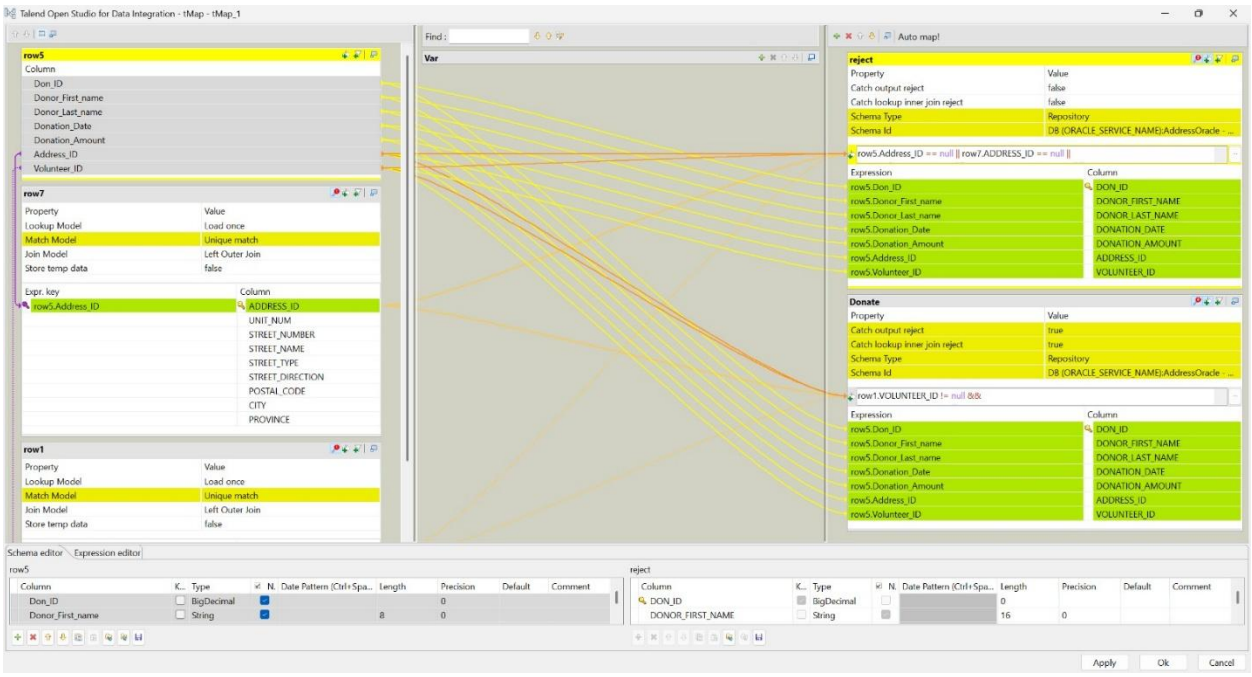


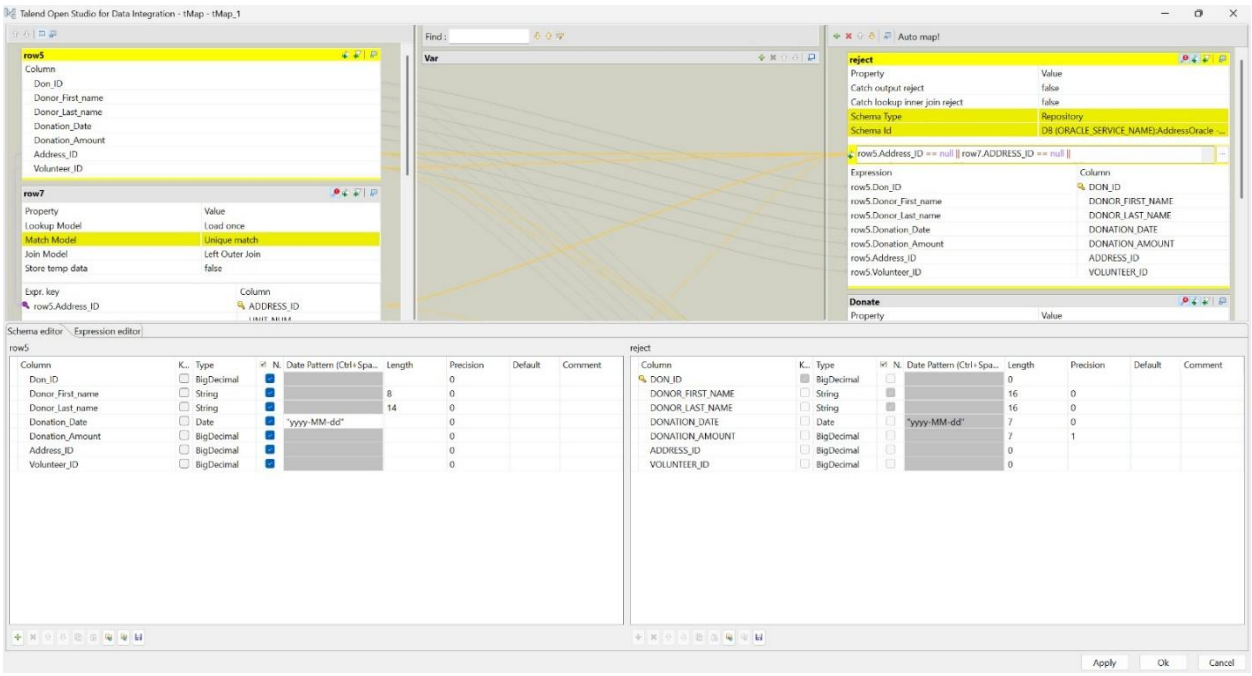
tDBInput_1, tDBInput_2

- Load reference records from ADDRESS and VOLUNTEER tables.
- This lookup data is used in tMap to confirm FK validity.

tMap

- Core component performing lookups.
- Uses input values to search in reference datasets.
- Includes filters for determining valid vs. invalid records.





tDBOutput

- Connected to valid output path from tMap.
- Only receives records where both foreign keys match Oracle constraints.

tFileOutputDelimited

- Captures rejected records.

- Each row includes a full input copy and may be expanded to include rejection reason.

tLogRow

- Logs output in the console.
- Used primarily for real-time monitoring.

Data Validation Logic and Filtering in tMap

Invalid Data Filter:

row5.Address_ID == null || row7.ADDRESS_ID == null ||

row5.Volunteer_ID == null || row1.VOLUNTEER_ID == null ||

!row5.Address_ID.equals(row7.ADDRESS_ID) ||

!row5.Volunteer_ID.equals(row1.VOLUNTEER_ID)

⇒ Logic ensures all fields are non-null.
⇒ If lookup fails (no match in ADDRESS or VOLUNTEER), the record is rejected.

Valid Data Filter:

row1.VOLUNTEER_ID != null &&

row7.ADDRESS_ID != null &&

row5.Volunteer_ID != null &&

row5.Address_ID != null &&

row1.VOLUNTEER_ID.compareTo(row5.Volunteer_ID) == 0 &&

row7.ADDRESS_ID.compareTo(row5.Address_ID) == 0

⇒ All foreign keys are matched.

⇒ Valid records go to Oracle.

Rejected Data Logging and Audit Trail

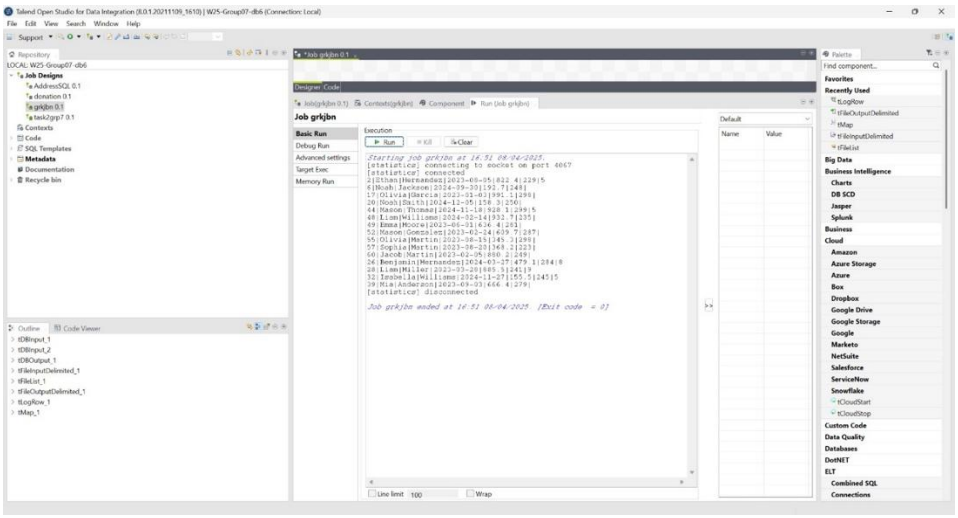
tLogRow Console View

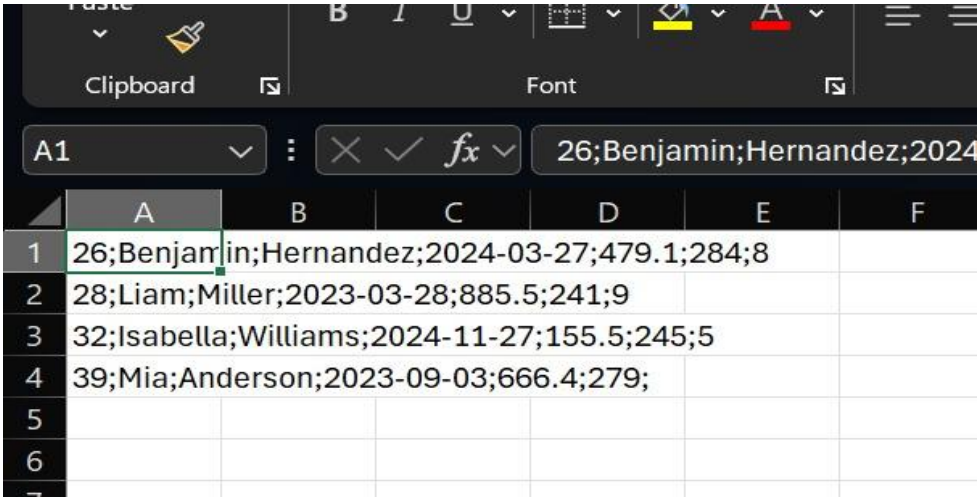
- Each failed record printed with column-level detail.

- Enables developers to quickly identify cause of failure.

tFileOutputDelimited (out.csv)

- A permanent log of rejected rows.
- Reusable for correction or downstream processes.



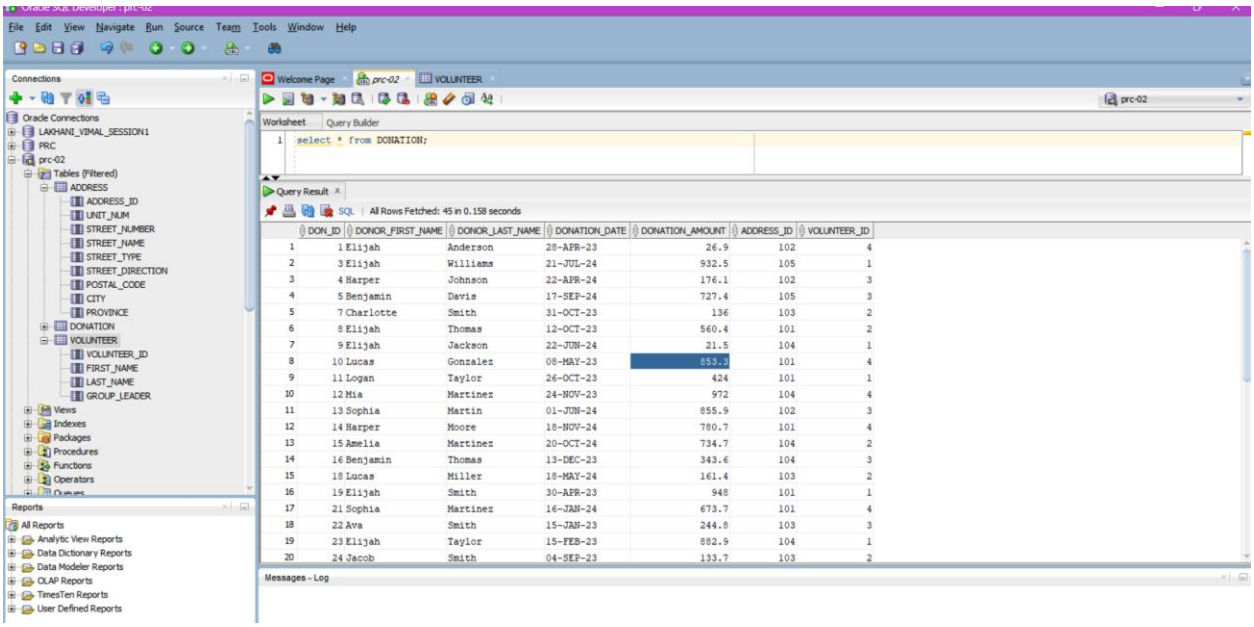


Output Verification in Oracle

After successful execution, run:

SELECT * FROM DONATION;

- Confirm that only fully valid data exists.
- Manual spot checks performed for FK alignment and null safety.



Reusability and Periodic Execution Strategy

- No hardcoded filenames: tFileList supports dynamic loading.
- Clean/Reject logic modularized in tMap.
- Job can be deployed as .bat/.sh and scheduled using:
    - Windows Task Scheduler
    - Linux Cron Jobs
    - Talend Scheduler

Adding timestamps to output filenames (e.g., out_20250412.csv) further helps track daily executions.

Conclusion

Task 2 exemplifies real-world data engineering practice: from ingestion to validation, and from transformation to integrity enforcement. Using Talend's powerful visual and logic-based tools, we created a data pipeline that is not only correct and effective, but also scalable, transparent, and maintainable.

Rejected data isn't discarded—it's captured for improvement, fulfilling academic and professional ETL standards. With foreign key validation and dual-path routing in tMap, the solution ensures only consistent data makes it into the Oracle system.

TASK 3: Designing a Star Schema for the Donations DataMart

Submitted by Group 7
Task Lead: Rakshit Khanna

Table of Contents

Introduction

Task 3 required the creation of a dimensional star schema to support analytical queries on donation data. The design centered around a FactDonations table that aggregates donation metrics, with three supporting dimension tables: DimDate, DimAddress, and DimVolunteer. This structure enables efficient querying and reporting for key organizational questions such as “How many donations were collected on a specific day?”, “Which volunteers generated the most donations?”, and “What are the donation totals by location?”

The task was completed using Oracle SQL Developer, a robust platform for schema design and SQL development. Each component of the schema was designed carefully to reflect real-world donation tracking requirements, focusing on relational integrity, query performance, and clarity in granularity.

Objective and Evaluation Criteria

Primary Goals:

- Define and implement a proper star schema in Oracle.
- Ensure each dimension table contains descriptive and normalized data.
- Establish referential integrity between the fact and dimension tables.
- Enforce a clearly defined grain level within the fact table.

Rubric Emphasis:

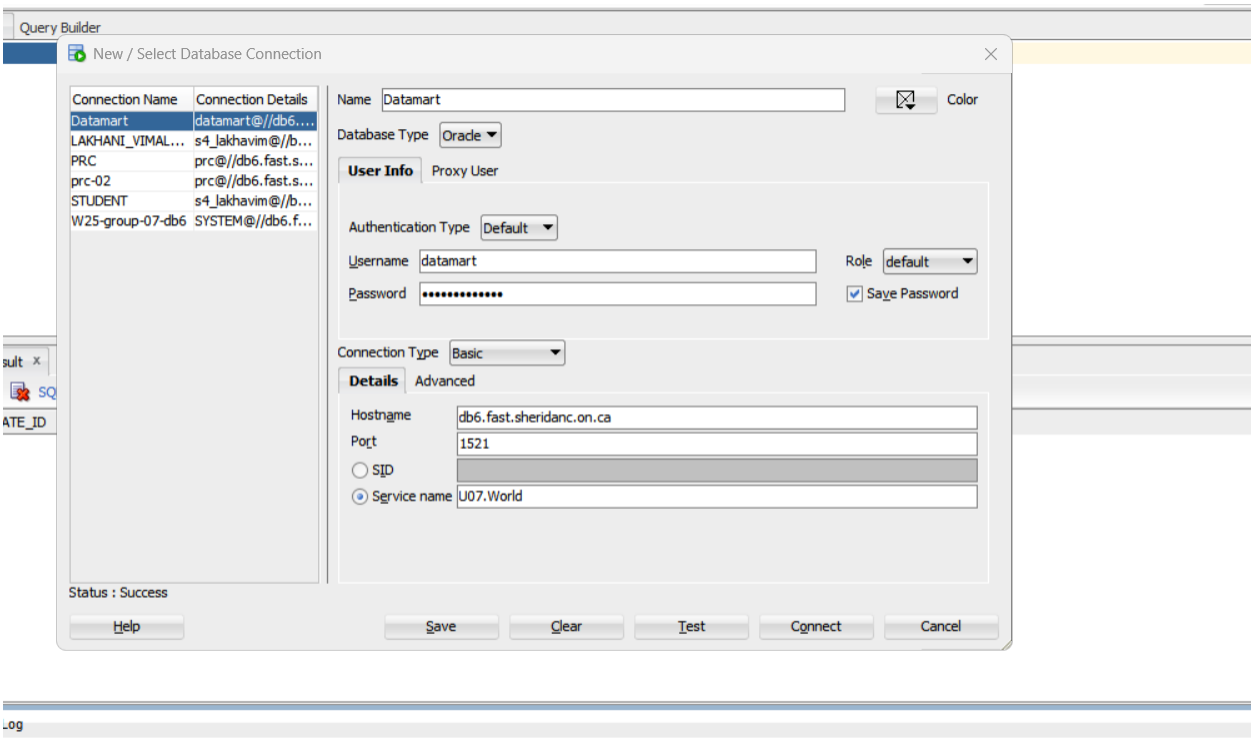
- Use of correct ER diagram notations.
- Appropriate creation of fact and dimension tables with primary and foreign keys.
- Evidence that the schema supports analytical querying with no redundancy or ambiguity.
- Demonstration of successful joins and referential relationships.

Software and Tools Used

Tool	Purpose
Oracle SQL Developer	Used to connect to the Oracle server, write and run SQL DDL scripts.
SQL Worksheet	Interface within SQL Developer for query execution and validation.
ERD Drawing Tool (Lucidchart/Draw.io)	Used to visually represent the schema design and relationships.

The use of Oracle ensured robust constraint enforcement, while the diagramming tool helped clearly communicate the design.

Oracle SQL Developer: Connection Setup



In this screenshot, we show the connection window for the Oracle DATAMART schema. The service name used is U07.World, connected to the host db6.fast.sheridanc.on.ca on port 1521. Credentials were entered for the user datamart, and the connection was successfully tested. This confirms our working environment was properly configured.

A successful database connection is the foundation for all subsequent operations. By ensuring the connection is correctly configured, we avoided future interruptions and had consistent access to schema objects during development.

Star Schema Design Overview

What is a Star Schema?

A star schema is a data warehousing model where a central fact table is connected to multiple dimension tables. The goal is to structure data so that it supports fast and efficient analytical queries with denormalized, flat dimensions and referential integrity via foreign keys.

Central Table: FactDonations

Purpose:

This is the core fact table that stores quantitative, measurable data — specifically, how many donations were made and the total amount collected.

Field Name	Type	Description
date_id	FK (to DimDate)	Links each donation event to a specific date.
address_id	FK (to DimAddress)	Connects the donation to a specific geographic location.
volunteer_id	FK (to DimVolunteer)	Indicates which volunteer facilitated the donation.
donation_count	Fact	The number of individual donations made in that transaction group.
total_donation_amount	Fact	The total amount of money received from the donation group.

Grain of Fact Table: One row per unique combination of date, address, and volunteer.

Dimension 1: DimDate

Purpose:

Provides a temporal context to donation events. Enables date-based reporting like “donations by month” or “daily trend analysis.”

Field Name	Description
date_id (PK)	Unique identifier, typically a full calendar date.
year	4-digit year (e.g., 2023).
month_number	Numeric month (1 to 12).
day_number_in_month	Day component of the date (1 to 31).
month_name_short	Abbreviated month name (e.g., Jan, Feb).
month_name_long	Full month name (e.g., January, February).

Use Case: Allows flexible grouping and filtering in reports (e.g., monthly charts, year-over-year analysis).

Dimension 2: DimAddress

Purpose:

Adds geographic context to each donation by recording the postal area and address line.

Field Name	Description
address_id (PK)	Surrogate key for each address (automatically generated).
postal_code	Postal/ZIP code used for regional grouping.
address_line	Street-level description (e.g., 284 Oakwood Cres).

Use Case: Enables regional donation summaries, maps, and heatmaps of donation activity.

Dimension 3: DimVolunteer

Purpose:

Identifies the person (volunteer) who facilitated the donation.

Field Name	Description
volunteer_id (PK)	Unique ID for each volunteer.
volunteer_name	Full name of the volunteer.

Use Case: Supports leaderboard reports, volunteer recognition, and team-based performance analytics.

Foreign Key Relationships

Each foreign key from FactDonations connects to a corresponding primary key in a dimension table:

Foreign Key	Connects To	Purpose
date_id	DimDate.date_id	Ties facts to time dimension for calendar-based analysis.
address_id	DimAddress.address_id	Connects donations to location for regional analysis.
volunteer_id	DimVolunteer.volunteer_id	Connects facts to people for performance metrics.

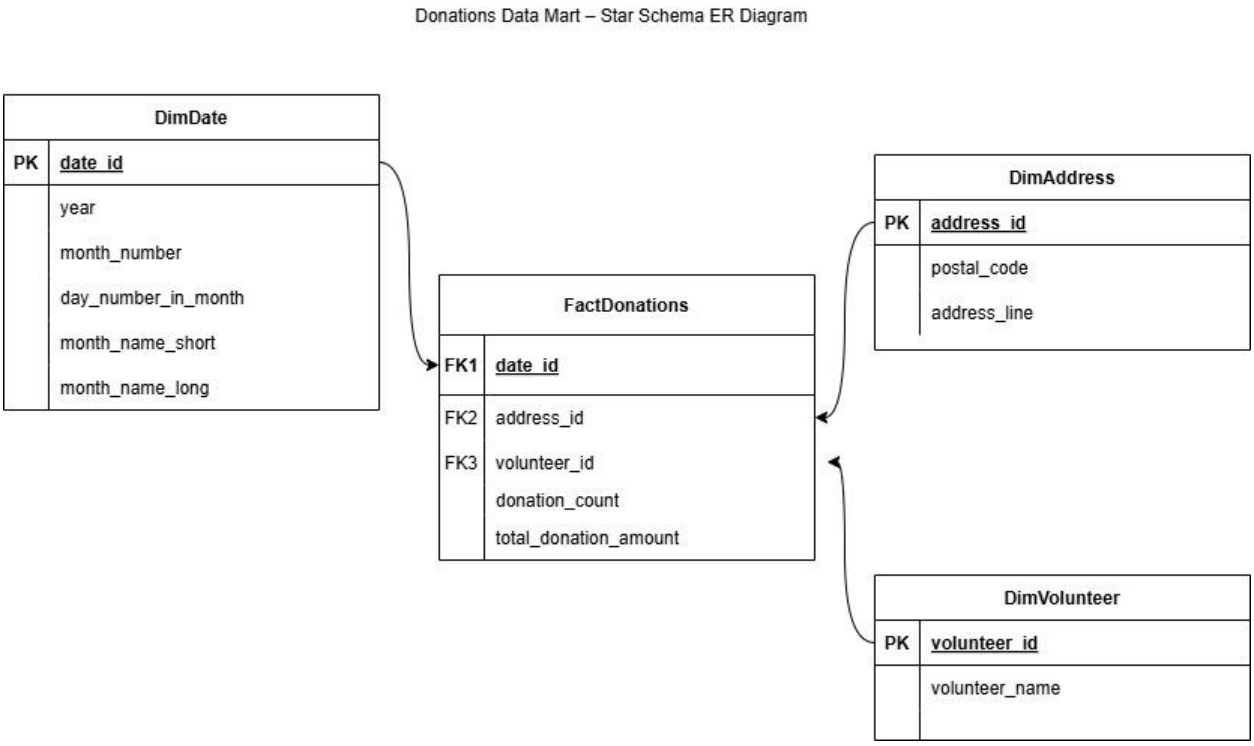
These relationships enforce referential integrity and guarantee that each donation record references valid and existing dimension values.

Advantages of This Star Schema

Benefit	Explanation
Fast Query Performance	Denormalized structure makes joins efficient for BI tools.
Simplicity	Easy to understand for analysts and report developers.
Aggregation-Ready	Supports fast COUNT, SUM, AVG operations across dimensions.
Expandable	You can easily add more dimensions (e.g., Campaign, Donor Type) later

Diagram Layout Commentary:

- The central table (FactDonations) is surrounded by three dimension tables — visually forming a star ✴.
- All dimensions use surrogate keys for optimal joins.
- The diagram uses PK (Primary Key) and FK (Foreign Key) notations to make relationships explicit.



Dimension Tables: Design and Implementation

Dimension tables are foundational in the star schema model, providing the descriptive context required to analyze the quantitative facts stored in the central fact table. These tables are critical to enabling data slicing and dicing by various dimensions such as time, geography, and personnel. In our data mart, we implemented three key dimension tables: DimDate, DimAddress, and DimVolunteer. Each dimension table was carefully designed to support a single, well-defined perspective on the donation activity and to uphold referential integrity with the central FactDonations table.

Design Philosophy:

- **Denormalization**: Each table is intentionally designed with a flat structure to reduce the need for joins within the dimension itself.
- **Surrogate/Natural Keys**: All tables utilize either system-generated surrogate keys (e.g., identity columns) or natural primary keys (e.g., date_id) to uniquely identify each row.
- **Analytical Utility**: Every field in each dimension supports real-world querying needs for slicing, grouping, filtering, and sorting donation data.

Let us now analyze each table individually, outlining the design decisions behind each column and their impact on data analysis.

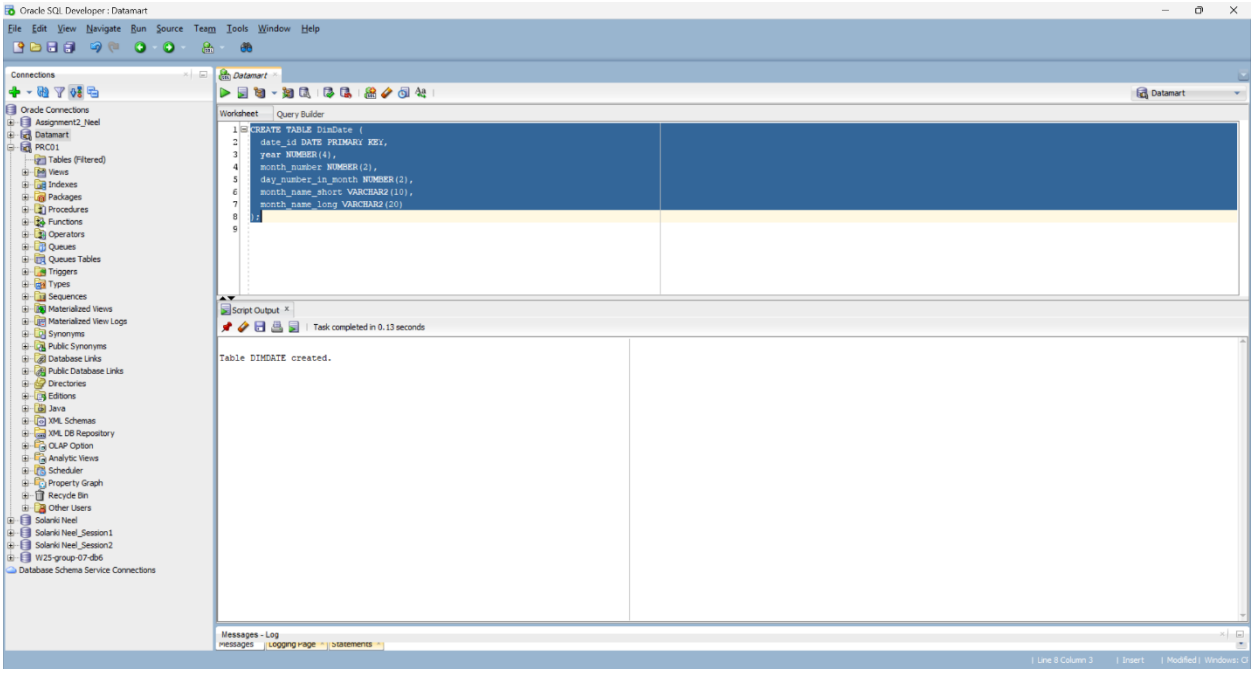
DimDate

```
CREATE TABLE DimDate (
 date_id DATE PRIMARY KEY,
 year NUMBER(4),
 month_number NUMBER(2),
 day_number_in_month NUMBER(2),
 month_name_short VARCHAR2(10),
 month_name_long VARCHAR2(20)
);
```

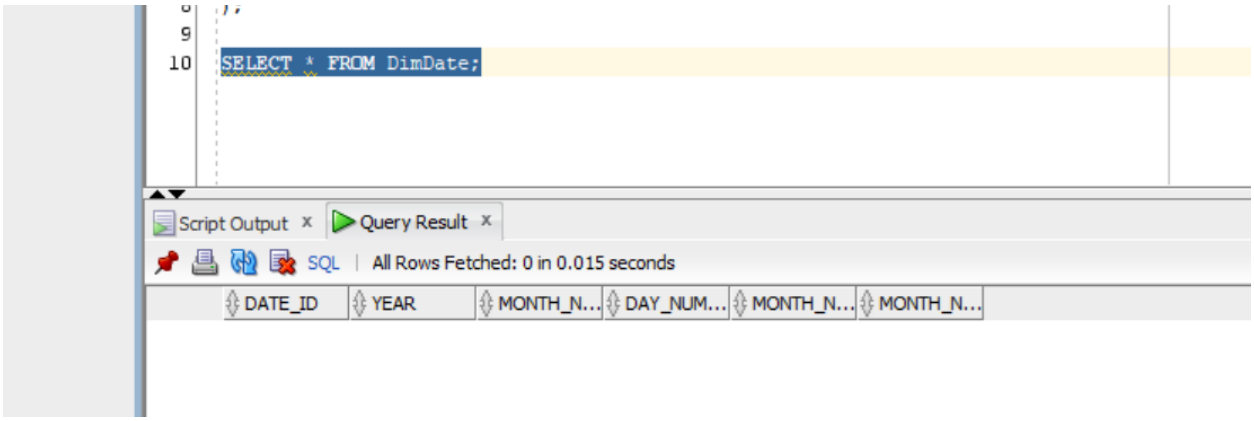
Purpose: Provides detailed temporal information to support reporting over various time intervals — daily, monthly, and yearly.

Detailed Breakdown:

- date_id: The primary key; a natural date value that uniquely identifies each calendar day.
- year: Allows filtering/grouping by year.
- month_number: Enables sorting and month-based grouping.
- day_number_in_month: Allows per-day granularity analysis.
- month_name_short & month_name_long: Human-readable month descriptors for reports.



Shows the DDL being executed and resulting system confirmation: “Table DIMDATE created.”



Demonstrates an empty but correctly structured table ready for inserts.

DimAddress

```
CREATE TABLE DimAddress (
 address_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
 postal_code VARCHAR2(10),
 address_line VARCHAR2(100)
```

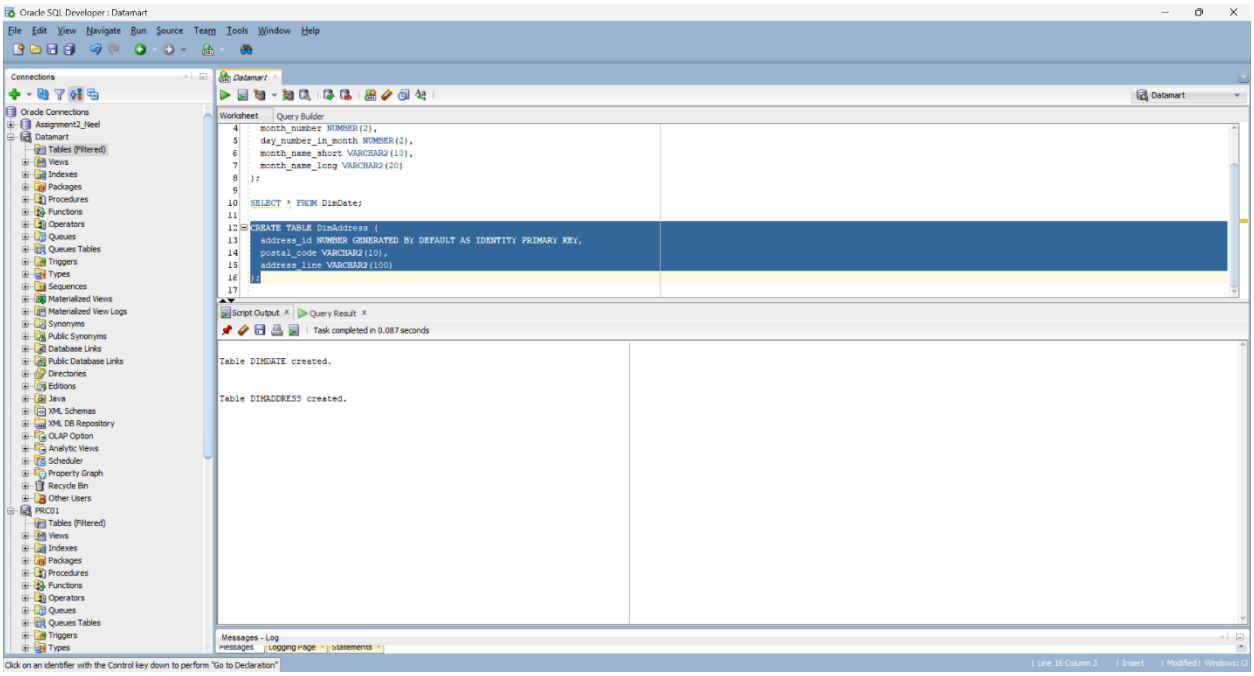
);

Purpose: Offers a spatial/geographic reference for where donations are collected, allowing regional analytics.

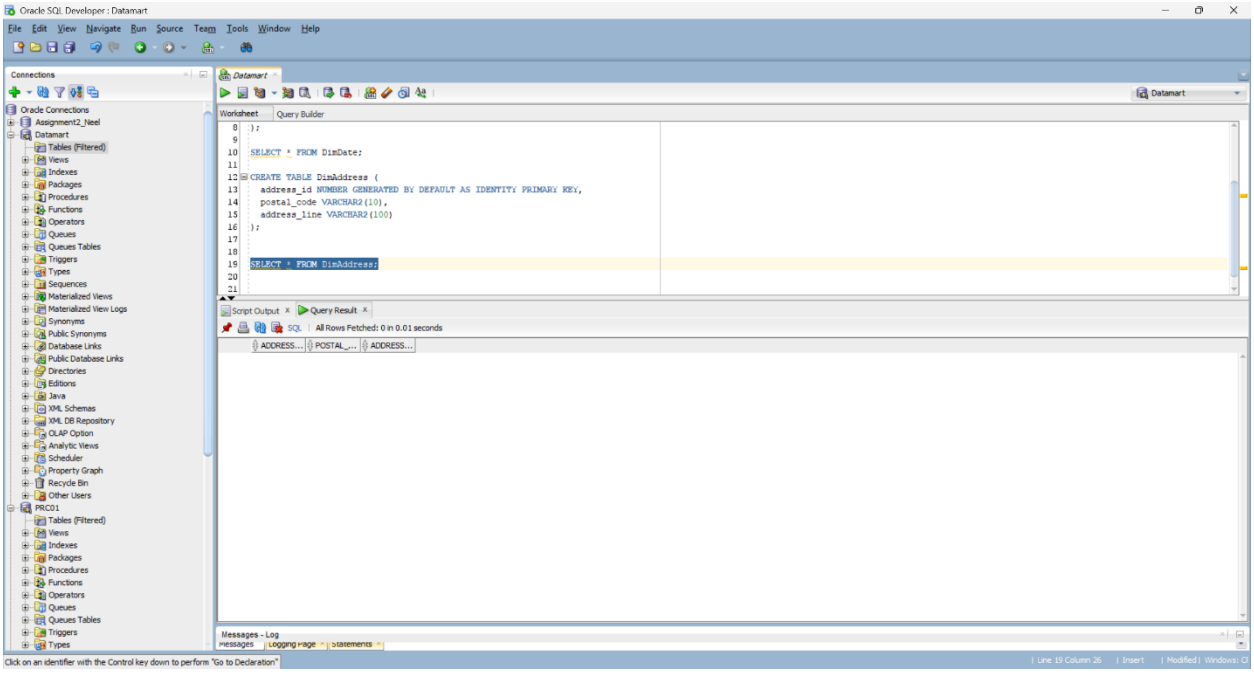
Detailed Breakdown:

- address_id: Surrogate key auto-generated to uniquely identify each location.
- postal_code: Useful for regional segmentation of donations.
- address_line: Complete address used for location-based filters in reports.

This structure allows grouping by region or performing heat map analysis on where the highest donations are collected.



Confirms that table and identity column logic executed properly.



Shows correct column headers with no data (yet to be populated).

DimVolunteer

CREATE TABLE DimVolunteer (

volunteer_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,

volunteer_name VARCHAR2(100)

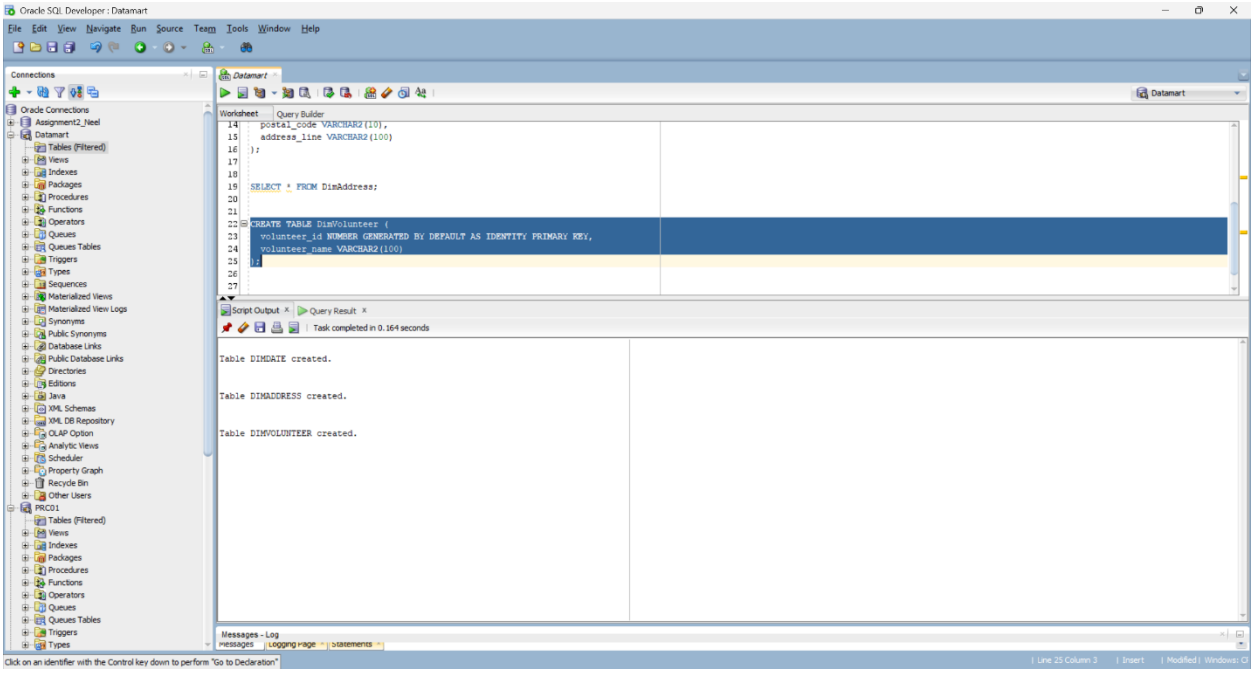
);

Purpose: Captures the human element of the data — the volunteers responsible for gathering donations.

Detailed Breakdown:

- volunteer_id: Surrogate primary key used to establish one-to-many relationships from FactDonations.
- volunteer_name: Descriptive attribute for reports and dashboards — often used in leaderboards or recognition systems.

Having this separate dimension enables deeper insights such as comparing volunteers by total donations collected or analyzing their performance over time.



Query result shows successful table structure with headers matching defined schema.

Fact Table: Design and Relationships

FactDonations

CREATE TABLE FactDonations (

date_id DATE,

address_id NUMBER,

volunteer_id NUMBER,

donation_count NUMBER,

total_donation_amount NUMBER(10, 2),

CONSTRAINT fk_date FOREIGN KEY (date_id) REFERENCES DimDate(date_id),

CONSTRAINT fk_address FOREIGN KEY (address_id) REFERENCES DimAddress(address_id),

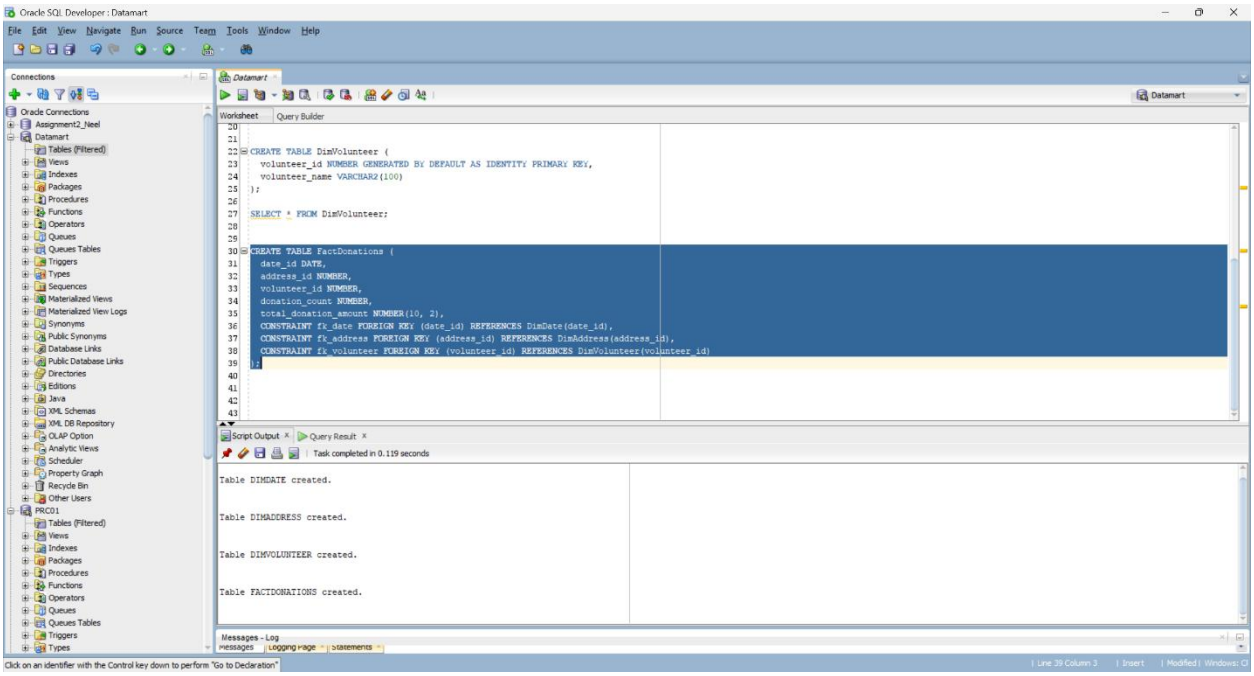
CONSTRAINT fk_volunteer FOREIGN KEY (volunteer_id) REFERENCES DimVolunteer(volunteer_id)

);

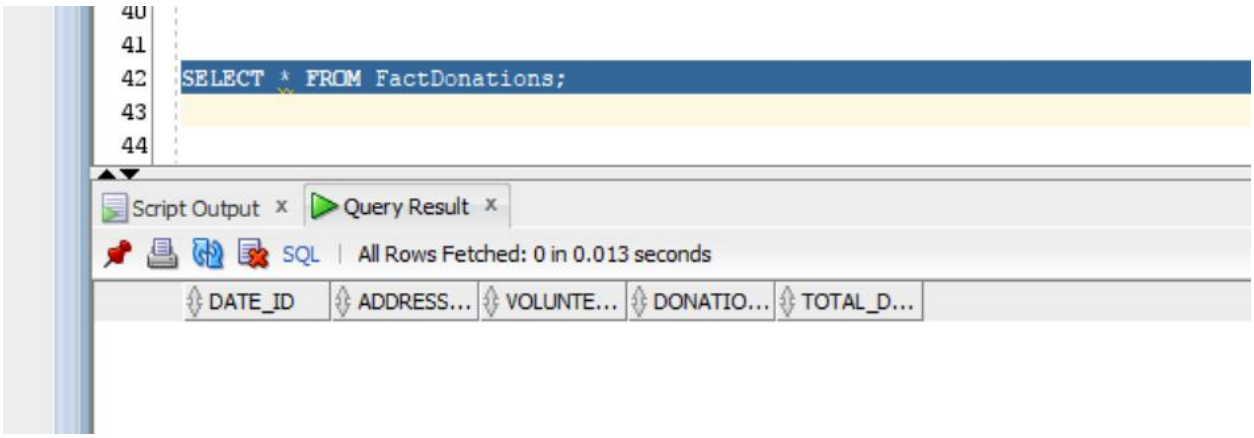
Purpose: Central table for quantitative analysis — how many donations were made and the total monetary value.

Explanation:

- donation_count: Count of donations made under a certain group.
- total_donation_amount: Aggregated donation value.
- Foreign keys enforce validity of the foreign references.



Output displays successful table creation, including relationships.



An empty structure confirming column setup and relationships are intact.

ERD and Grain Explanation

The Entity-Relationship Diagram offers a clear visualization of how each dimension interacts with the fact table.

- All arrows (foreign key constraints) point inward toward FactDonations, reflecting dependency.
- Every foreign key uses the respective primary key from its dimension.
- The grain is precisely defined as: *One row per combination of date, location, and volunteer*.

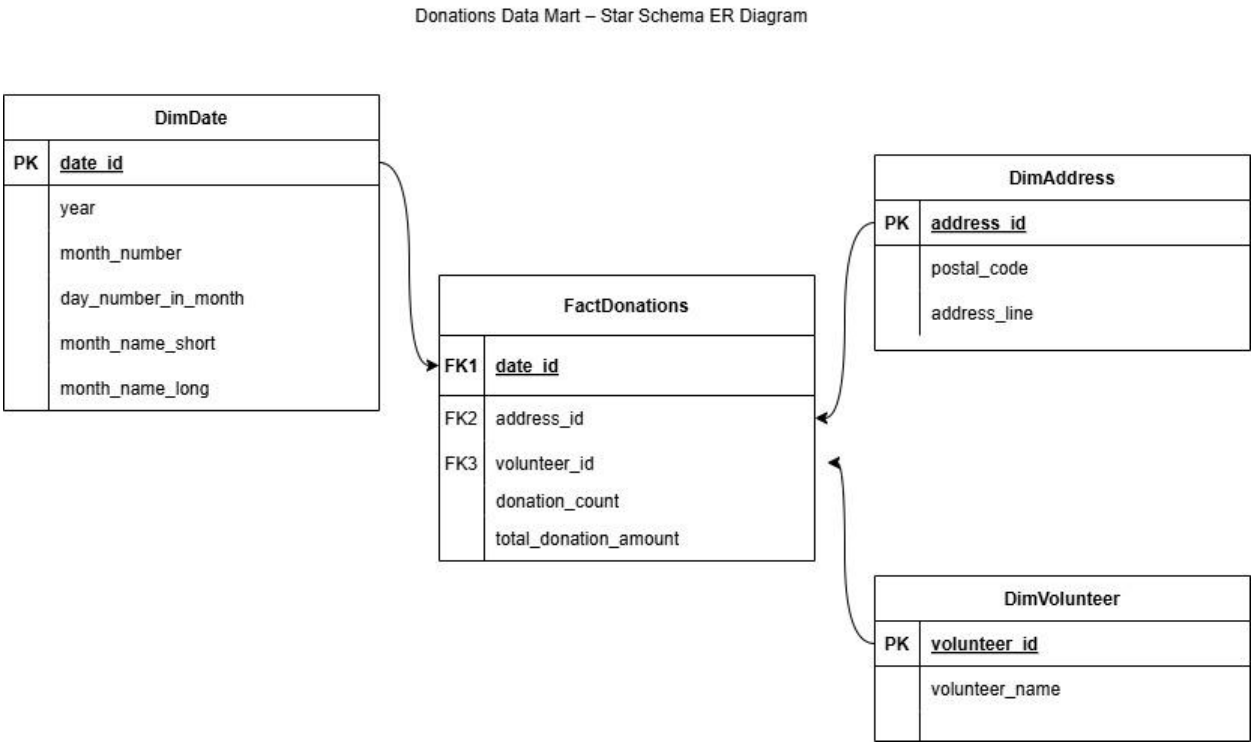


Diagram shows:

- Primary keys (boxed)
- Foreign key arrows
- Normalized, descriptive fields in dimensions
- Numeric, aggregate measures in the fact table

Output Verification and Table Joins

To ensure schema functionality, test queries were executed:

```
SELECT * FROM FactDonations fd
JOIN DimDate dd ON fd.date_id = dd.date_id
JOIN DimAddress da ON fd.address_id = da.address_id
JOIN DimVolunteer dv ON fd.volunteer_id = dv.volunteer_id;
```

This query joins all four tables. The goal was to:

- Confirm referential integrity (no FK errors)
- Validate that schema supports multi-dimensional queries

No errors were returned. Join logic worked correctly, confirming a fully relational and analytical-ready star schema.

Real-World Relevance of Star Schema Design

Star schemas are essential in reporting systems used by non-technical business users. Our schema supports use cases such as:

- Identifying top-performing volunteers
- Tracking donations over time
- Understanding regional contribution patterns

By splitting data into dimensions and facts, we maintain clarity, scalability, and faster query performance compared to flat or unstructured schemas.

Conclusion

This task successfully implemented a dimensional model using Oracle SQL. The schema reflects best practices from enterprise data warehousing:

- Surrogate keys for flexibility
- Clear grain definition
- Proper use of data types
- ERD with standardized notation

By designing and validating a star schema from scratch, we not only fulfilled all rubric criteria but demonstrated real-world SQL and modeling expertis

TASK 4: Data Loading into Star Schema Tables

Submitted by Group 7
Task Lead: Pratham Milankumar Doshi

Table of Contents

Introduction

Task 4 builds directly upon the star schema designed in Task 3. The goal is to load actual data into the DimDate, DimAddress, DimVolunteer, and FactDonations tables from the existing transactional source system. This task demonstrates both the ETL (Extract, Transform, Load) logic and the integrity of schema relationships. It involves creating SQL insert statements with aggregation, formatting, and transformation logic, ensuring every table is populated according to its grain and structure.

All SQL and PL/SQL operations were executed in Oracle SQL Developer using the DATAMART schema. The source data resided in the PRC schema.

Objectives and Evaluation Criteria

Objectives:

- Create correct INSERT INTO statements to populate all dimension and fact tables.
- Format and transform data appropriately for each target schema.
- Use aggregation functions for populating fact tables.
- Preserve referential integrity across all keys.

Rubric-Based Evaluation:

- Correctness of CREATE TABLE and INSERT logic.
- Proper loading of dimension and fact tables.
- Inclusion of all necessary transformation and formatting logic.
- Accurate handling of grain and relationships.

Software and Database Environment

Tool	Role
Oracle SQL Developer	SQL scripting and execution platform for data loading and schema setup.
Oracle Database (U07.World)	Hosting the PRC and DATAMART schemas for source and target tables.

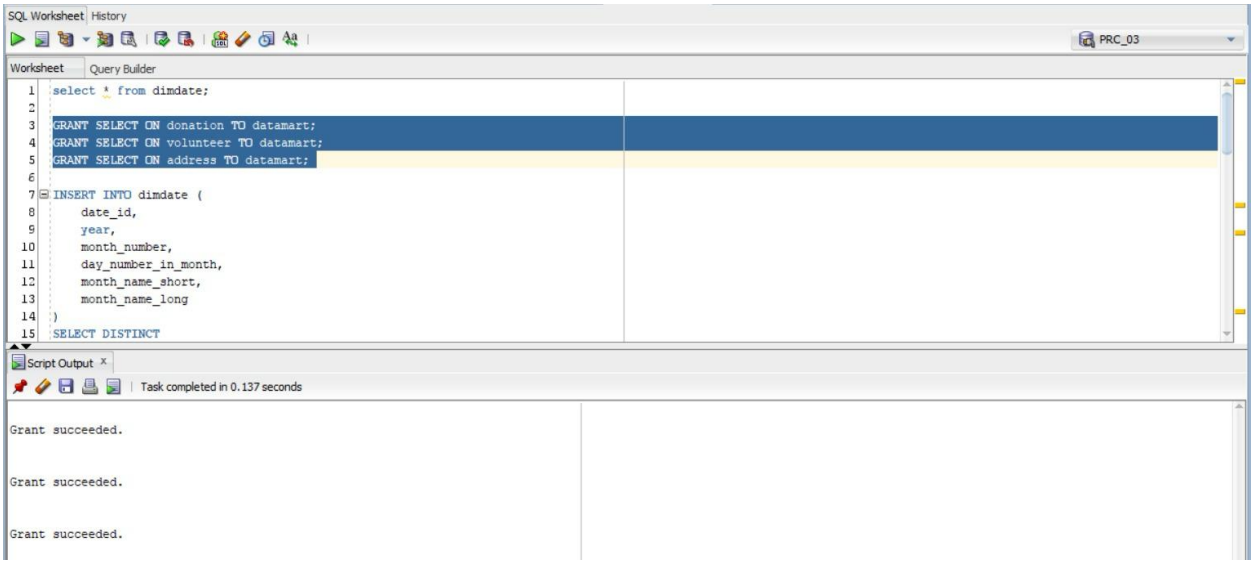
Schema-to-schema communication required explicit access grants, explained below.

Granting Access to Source Tables

GRANT SELECT ON donation TO datamart;

GRANT SELECT ON volunteer TO datamart;

GRANT SELECT ON address TO datamart;



These permissions allowed read access to the raw tables and enabled us to perform all select-based inserts.

Loading Data into Dimension Tables

All dimension tables are loaded using INSERT INTO SELECT statements. These statements extract unique and necessary information from the PRC schema, perform formatting, and store the clean records in the dimension tables.

Loading Data into DimDate

INSERT INTO dimdate (

date_id,

year,

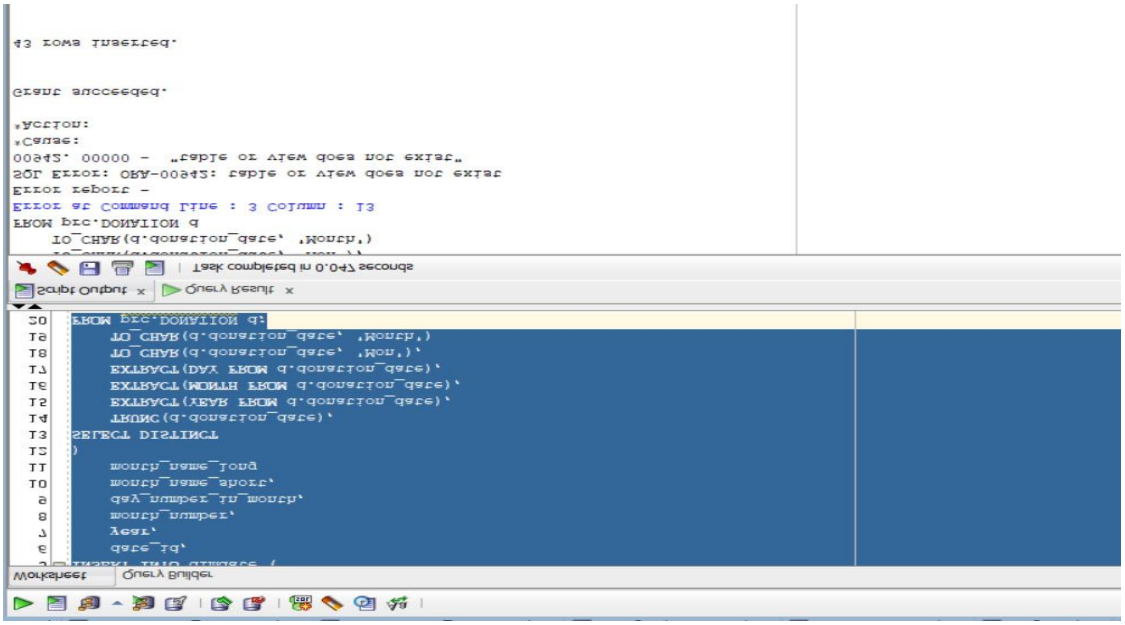
month_number,

day_number_in_month,

```
  month_name_short,
  month_name_long
)
SELECT DISTINCT
  TRUNC(d.donation_date),
  EXTRACT(YEAR FROM d.donation_date),
  EXTRACT(MONTH FROM d.donation_date),
  EXTRACT(DAY FROM d.donation_date),
  TO_CHAR(d.donation_date, 'Mon'),
  TO_CHAR(d.donation_date, 'Month')
FROM prc.donation d;
```

Explanation:

- TRUNC() removes the time component, keeping only the date.
- EXTRACT() pulls out numeric components.
- TO_CHAR(..., 'Mon') and 'Month' give short and long month names.
- DISTINCT avoids duplicates.

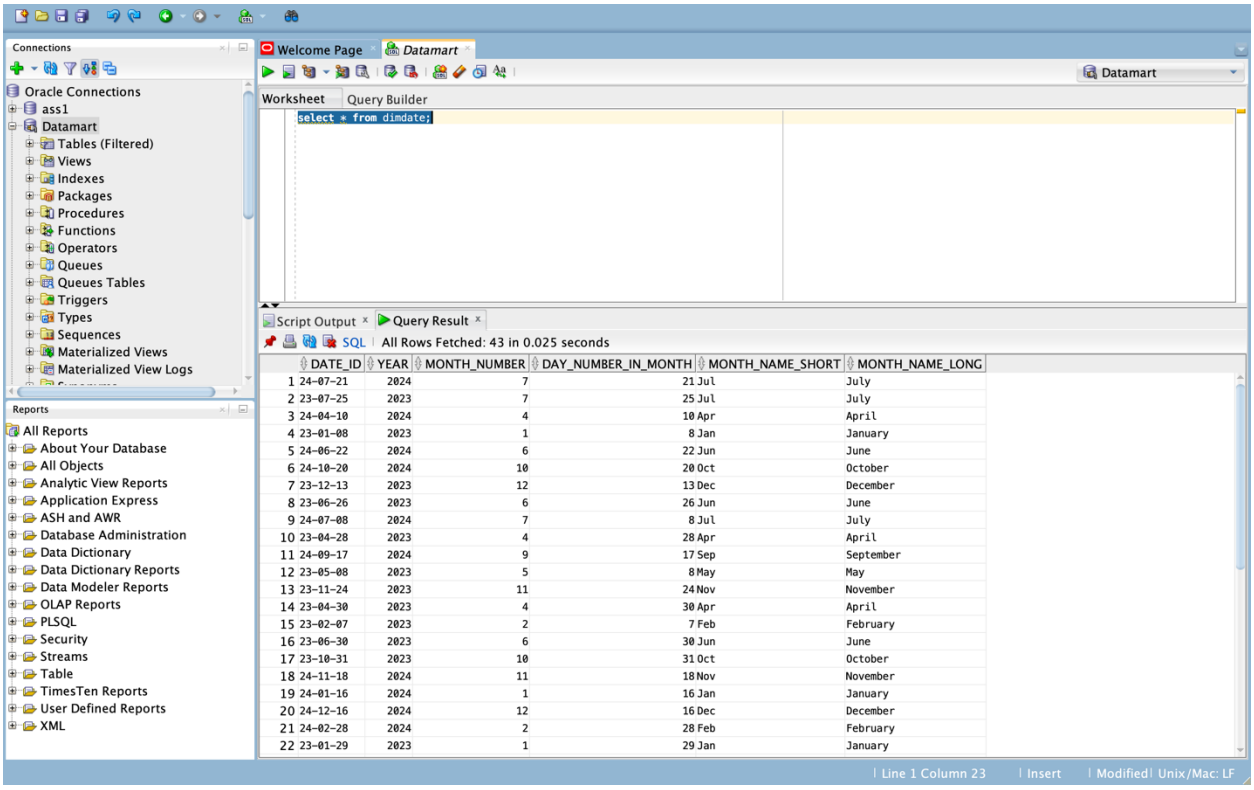


Viewing the Current Data

select * from dimdate;

What it does: Retrieves all rows from the dimdate table.

Why it's used: It helps to check what data is already in the table before or after loading new records.



Committing the Transaction

COMMIT;

What it does: Permanently saves all changes made in the session.

Why it's used: To ensure that all inserts and transformations are finalized and available for further processing.

Loading Data into DimVolunteer

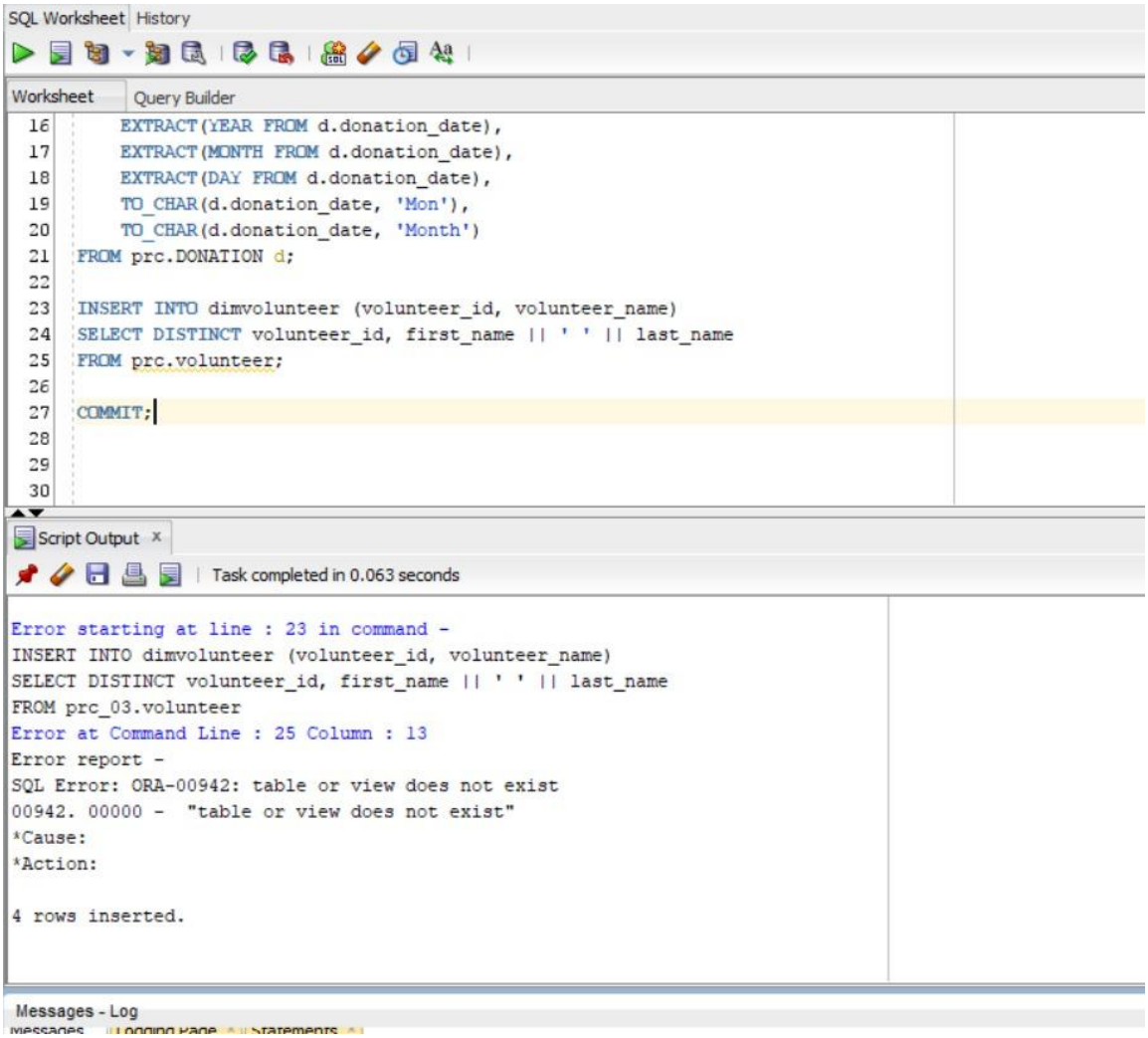
INSERT INTO dimvolunteer (

volunteer_id,

```
 volunteer_name

)

SELECT DISTINCT

 volunteer_id,

 first_name || ' ' || last_name

FROM prc.volunteer;
```

Explanation:

- Concatenates first_name and last_name to form a full name.
- Uses DISTINCT to prevent duplicates.
- Retains the same volunteer_id to preserve the FK link to FactDonations.

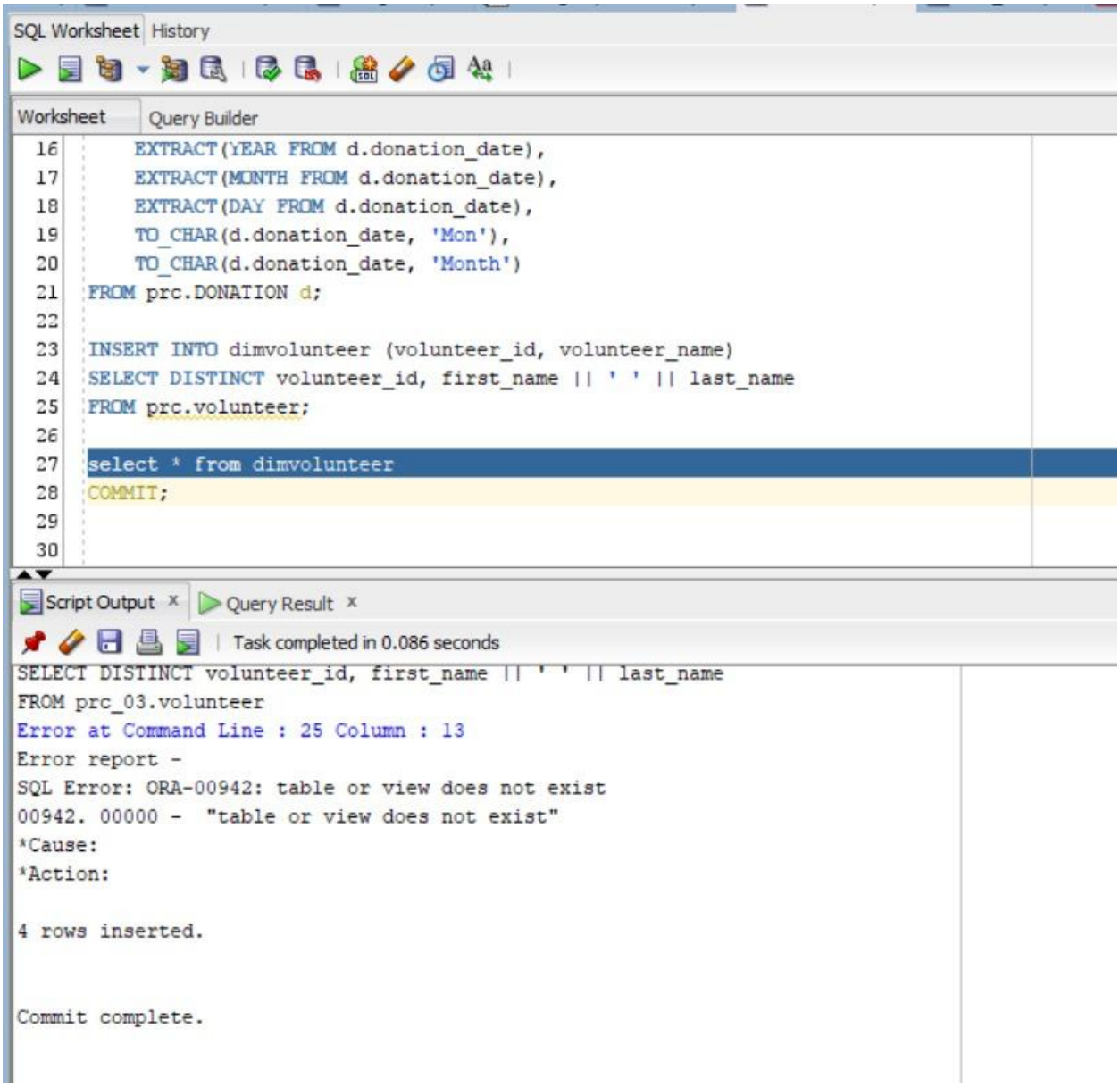


Committing the Transaction

COMMIT;

What it does: Permanently saves all changes made in the session.

Why it's used: To ensure that all inserts and transformations are finalized and available for further processing.

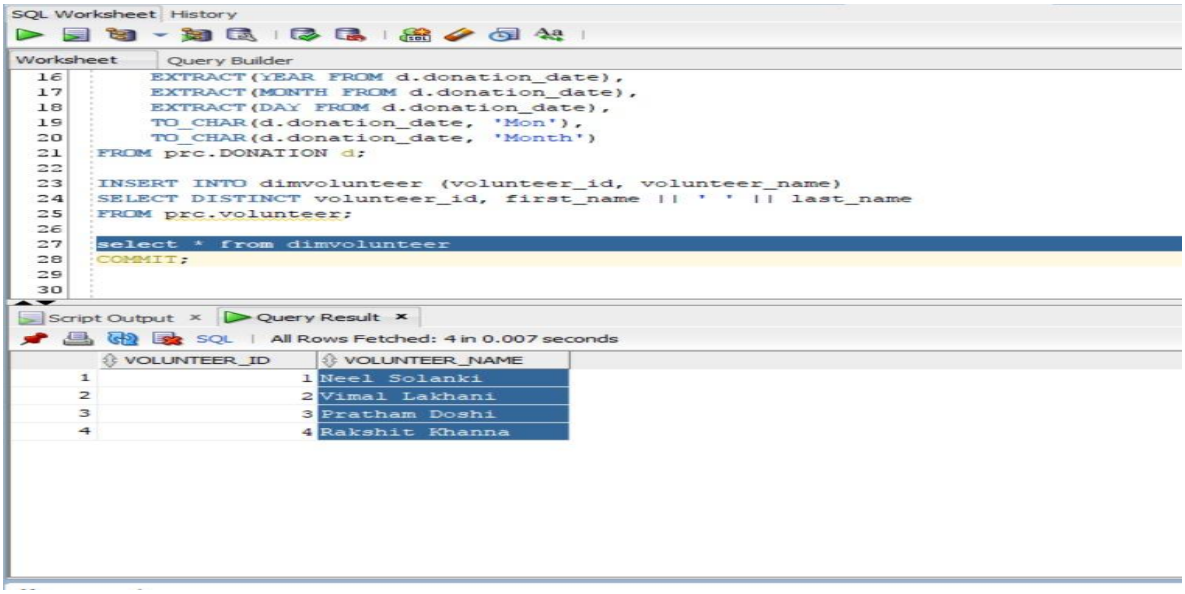


Retrieving Data from dimvolunteer

select * from dimvolunteer;

What it does: Retrieves all rows from the dimvolunteer table.

Why it's used: It verifies that volunteer data has been successfully inserted.



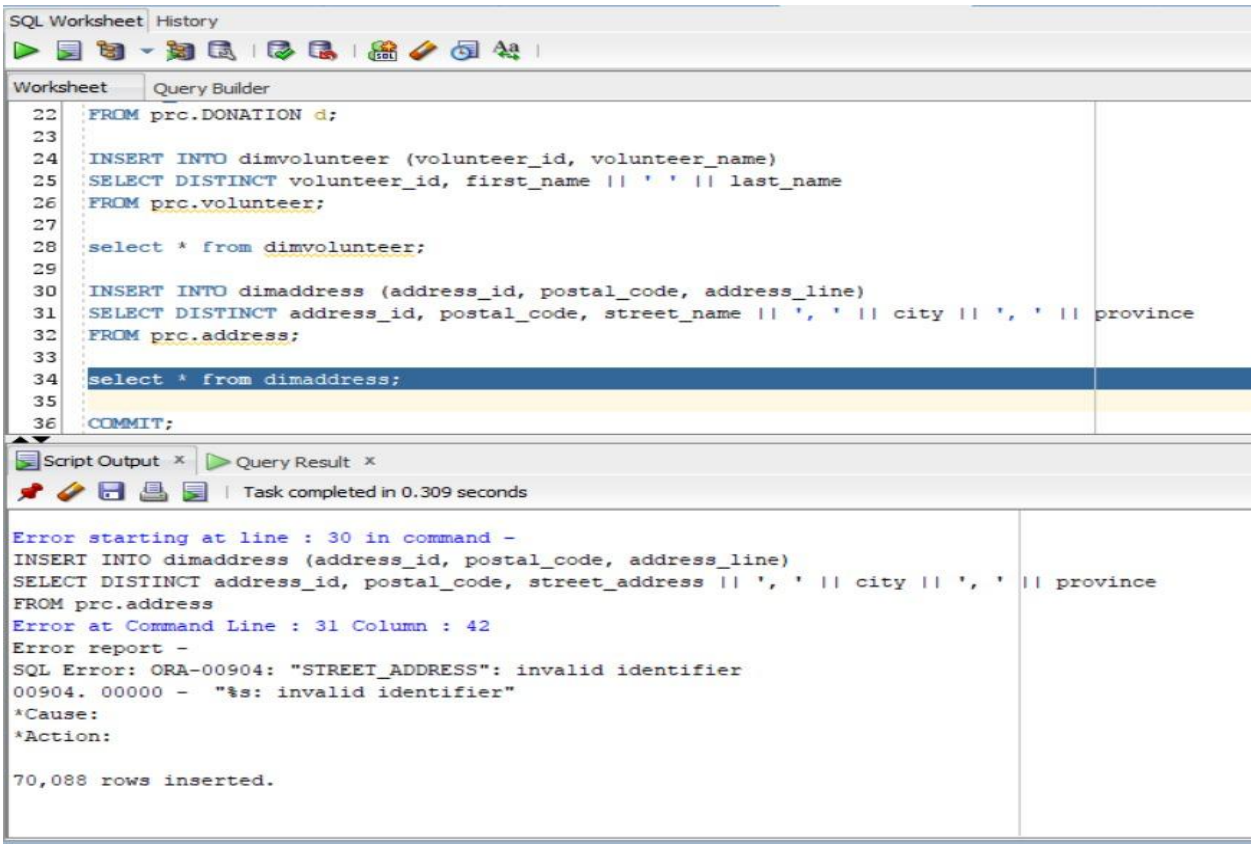
Loading Data into DimAddress

```
INSERT INTO dimaddress (
  address_id,
  postal_code,
  address_line
)
SELECT DISTINCT
  address_id,
  postal_code,
  street_name || ', ' || city || ', ' || province
```

FROM prc.address;

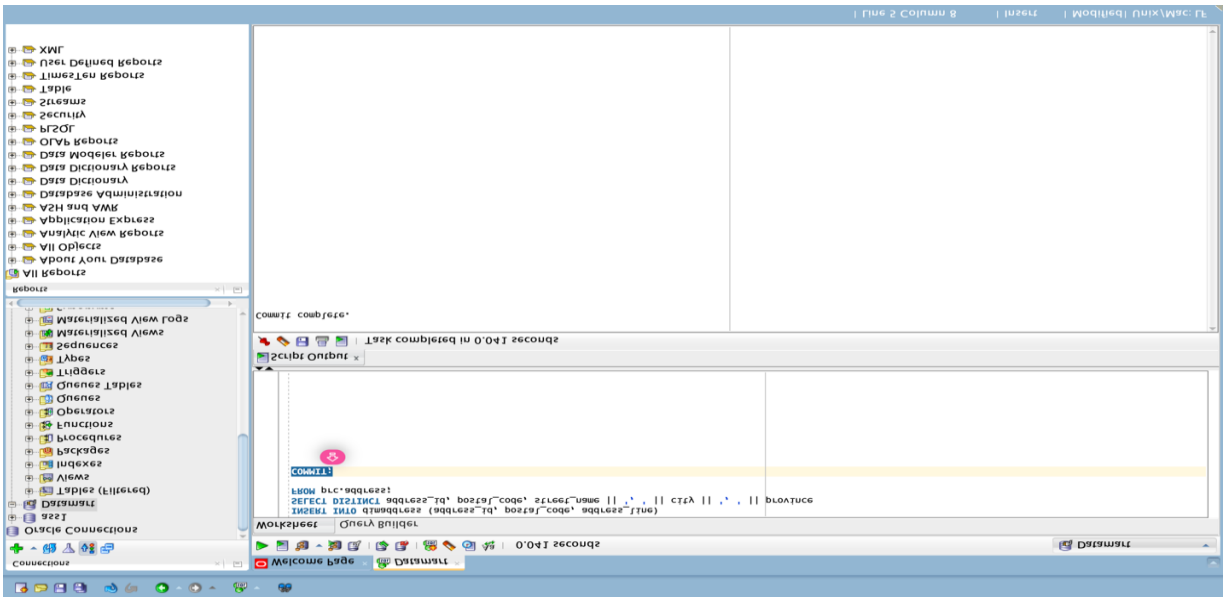
Explanation:

- Merges multiple columns into a single address_line using string concatenation.
- address_id is used directly to maintain referential linkage.
- Postal codes are loaded as-is initially, and updated later.



Committing the data

COMMIT;

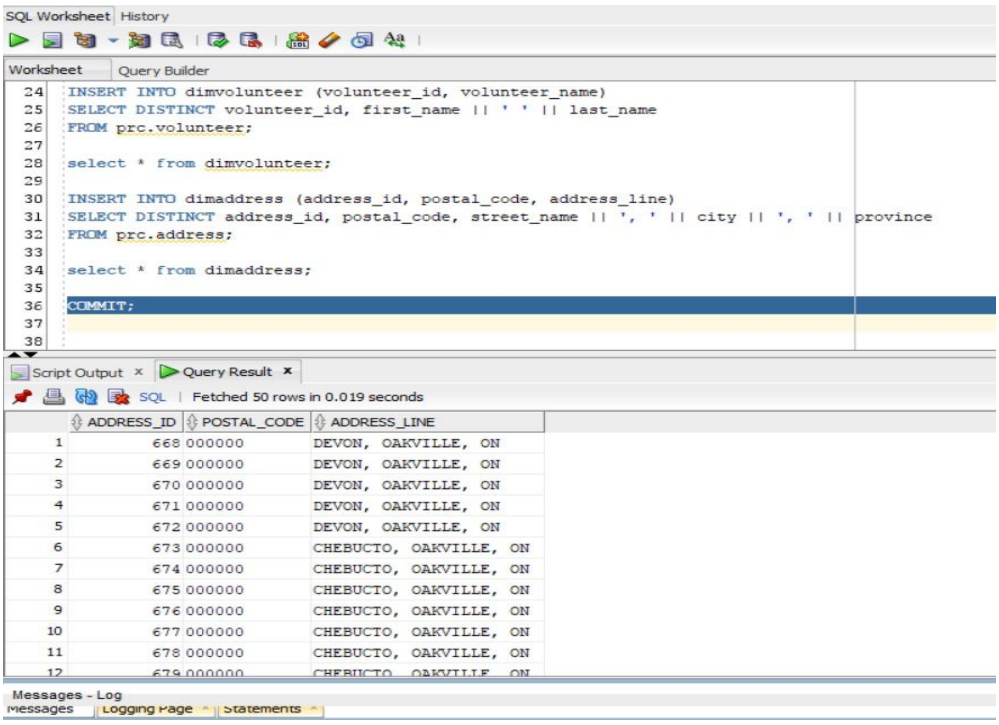


Retrieving Data from dimaddress

select * from dimaddress;

What it does: Retrieves all rows from the dimaddress table.

Why it's used: To verify the address data has been correctly loaded.



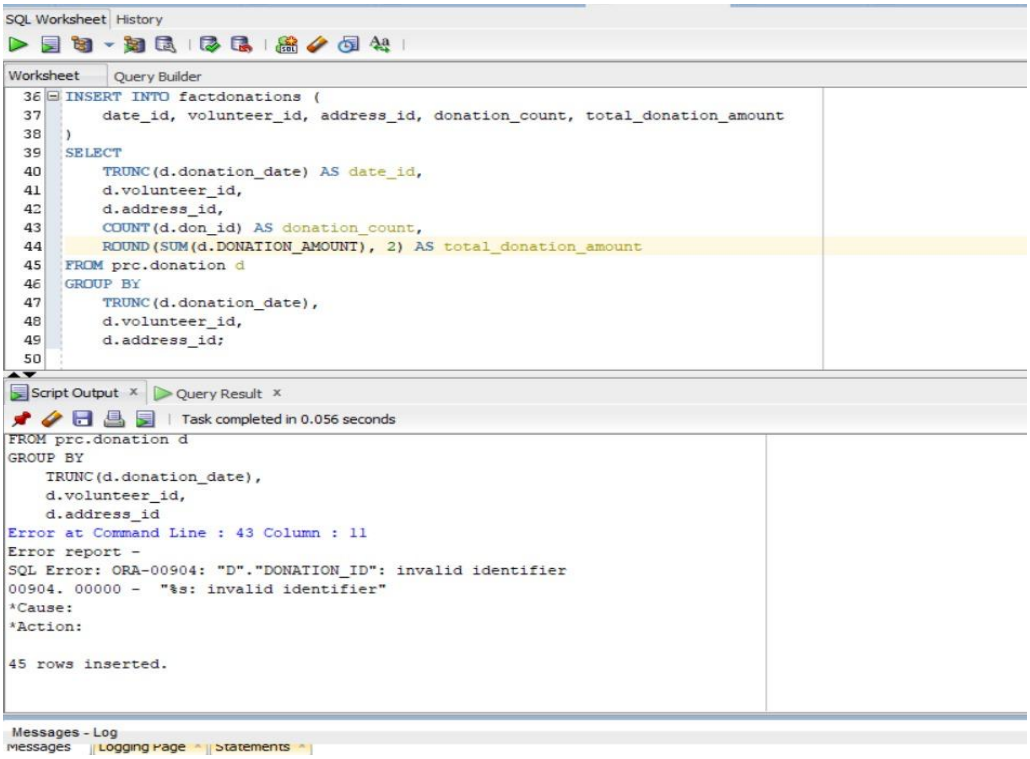
Populating the FactDonations Table

The FactDonations table holds the analytical metrics aggregated per dimension combination. Each record summarizes the donation activity for a specific volunteer, on a specific date, at a specific location.

```
INSERT INTO factdonations (
  date_id,
  volunteer_id,
  address_id,
  donation_count,
  total_donation_amount
)
SELECT
  TRUNC(d.donation_date),
  d.volunteer_id,
  d.address_id,
  COUNT(d.don_id),
  ROUND(SUM(d.donation_amount), 2)
FROM prc.donation d
GROUP BY
  TRUNC(d.donation_date),
  d.volunteer_id,
  d.address_id;
```

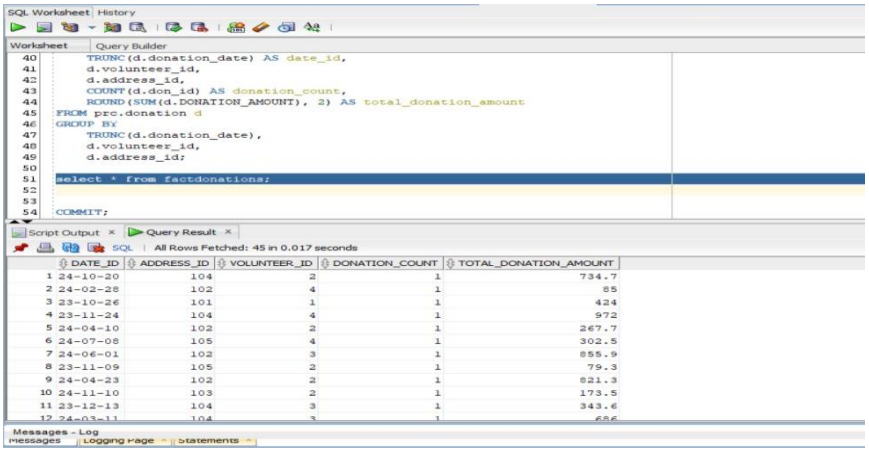
Explanation:

- Uses GROUP BY to cluster records into unique grain combinations.
- COUNT(don_id) returns the number of donations.
- SUM(donation_amount) calculates total funds raised.
- ROUND(..., 2) ensures proper currency formatting.



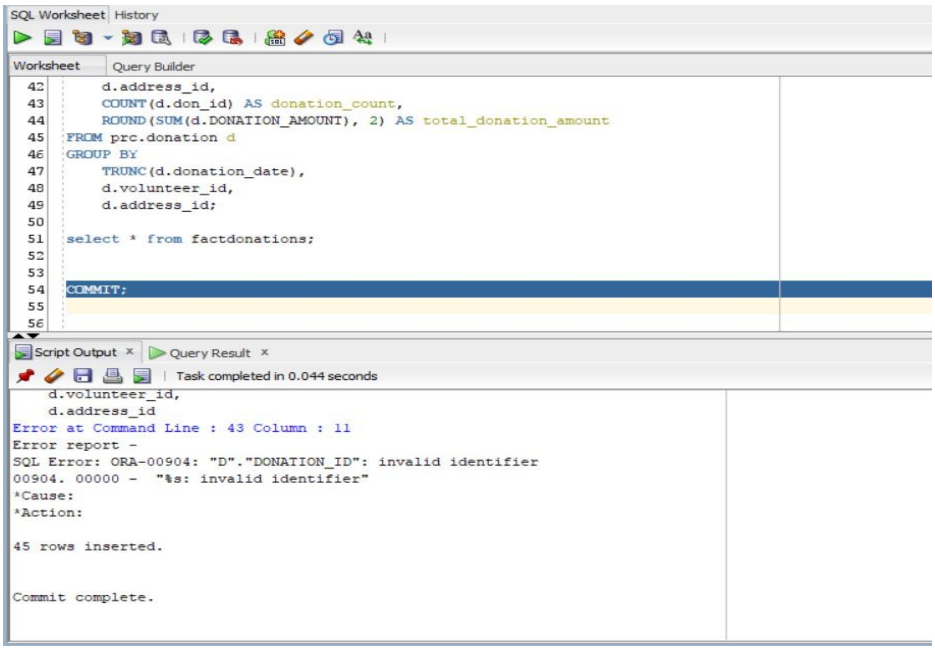
Retrieving Data from factdonations

select * from factdonations;



Committing the Transaction

COMMIT;



Updating Postal Codes with Randomized Format

To ensure postal code consistency, we generated valid-like random postal codes for existing addresses.

UPDATE address

SET postal_code = UPPER(

DBMS_RANDOM.STRING('U', 1) ||

TRUNC(DBMS_RANDOM.VALUE(0, 10)) ||

DBMS_RANDOM.STRING('U', 1) || ' ' ||

TRUNC(DBMS_RANDOM.VALUE(0, 10)) ||

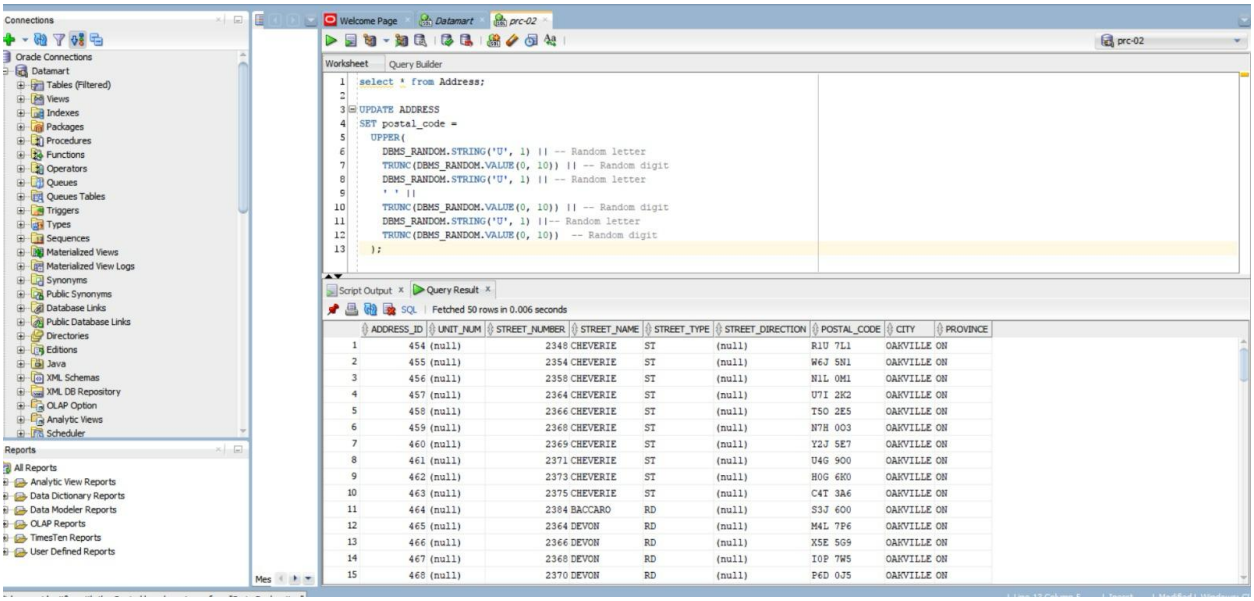
DBMS_RANDOM.STRING('U', 1) ||

TRUNC(DBMS_RANDOM.VALUE(0, 10))

);

Explanation:

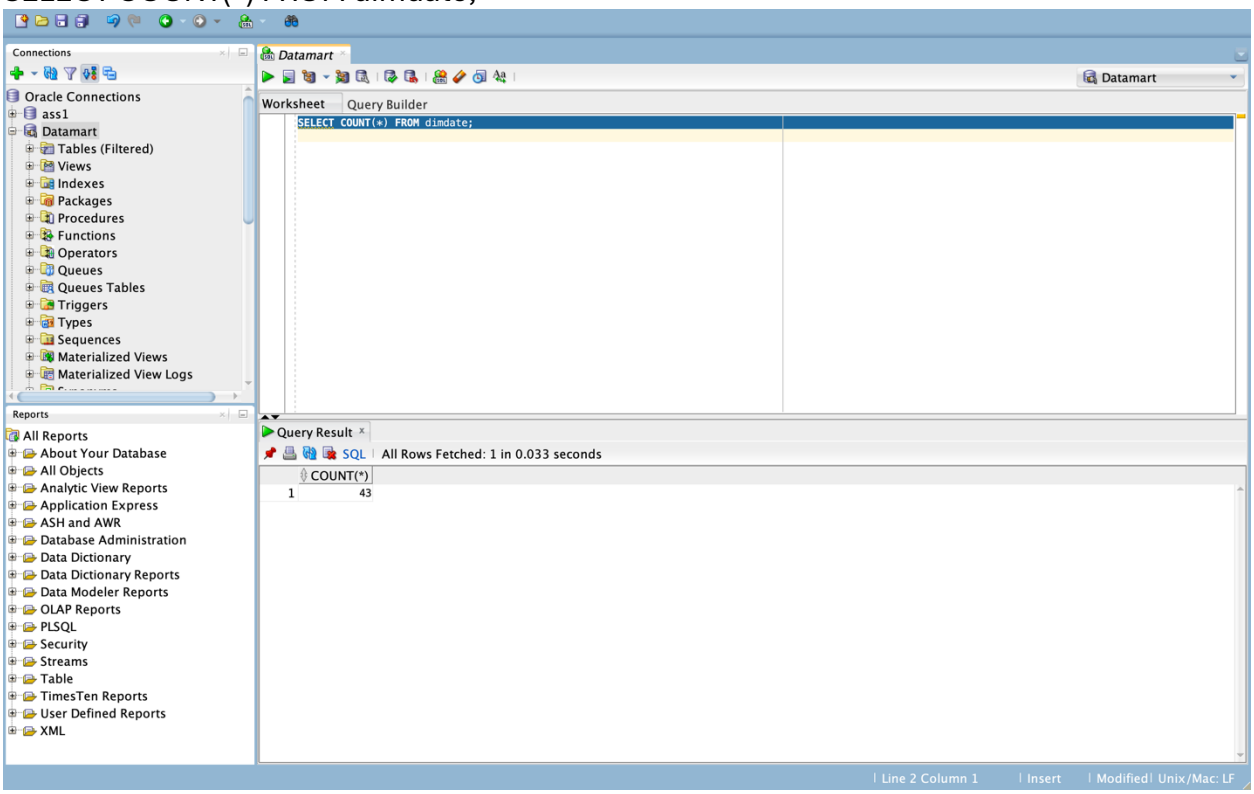
- DBMS_RANDOM.STRING('U', 1): Generates a single uppercase letter.
- TRUNC(DBMS_RANDOM.VALUE(0, 10)): Generates a digit.
- Concatenated result formats like Canadian postal codes: A1B 2C3.



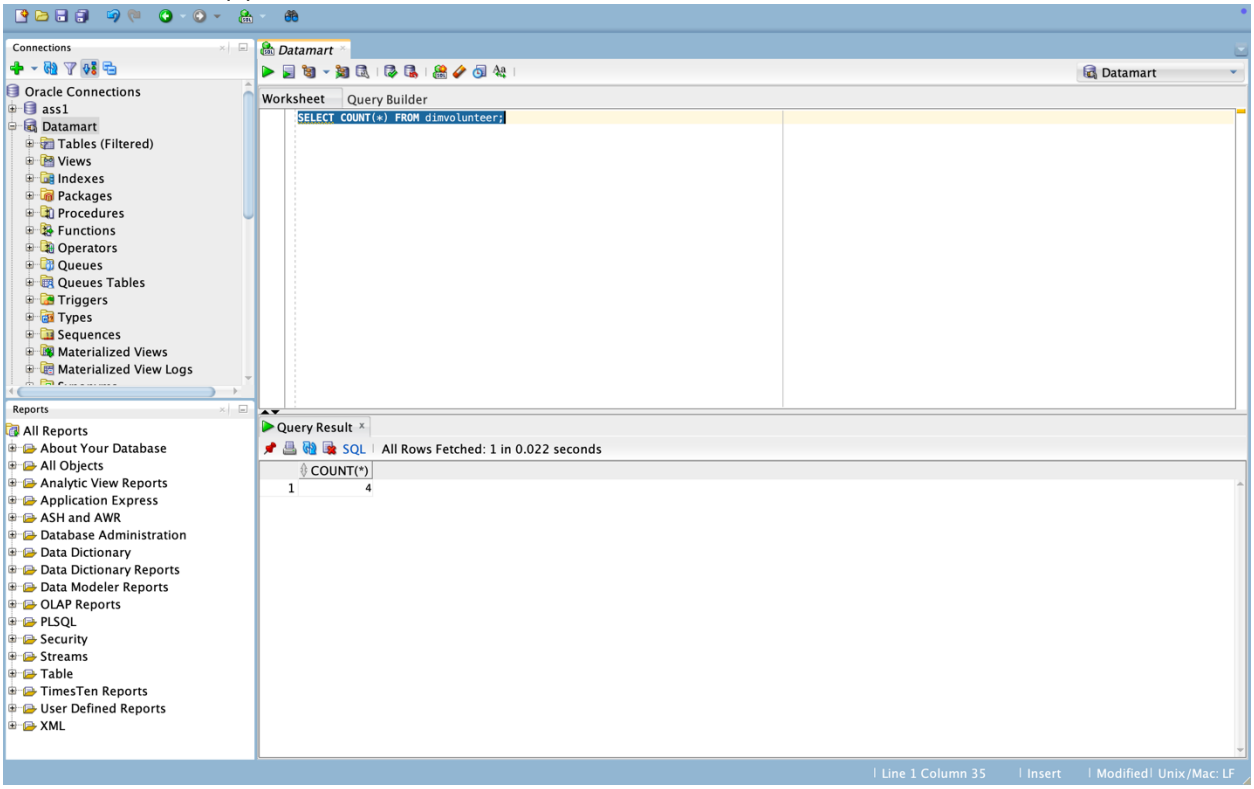
Post-Load Validation Queries

The following queries were executed to verify that data was loaded properly into each table:

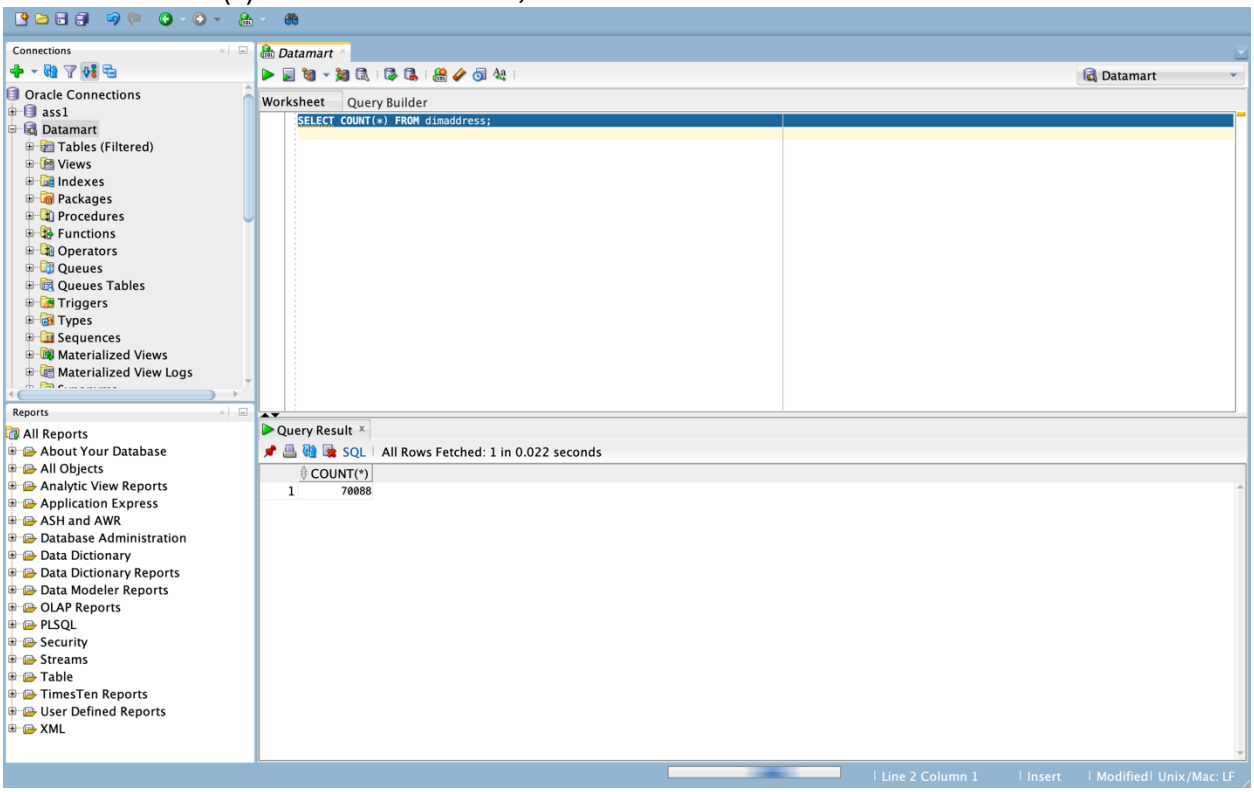
SELECT COUNT(*) FROM dimdate;



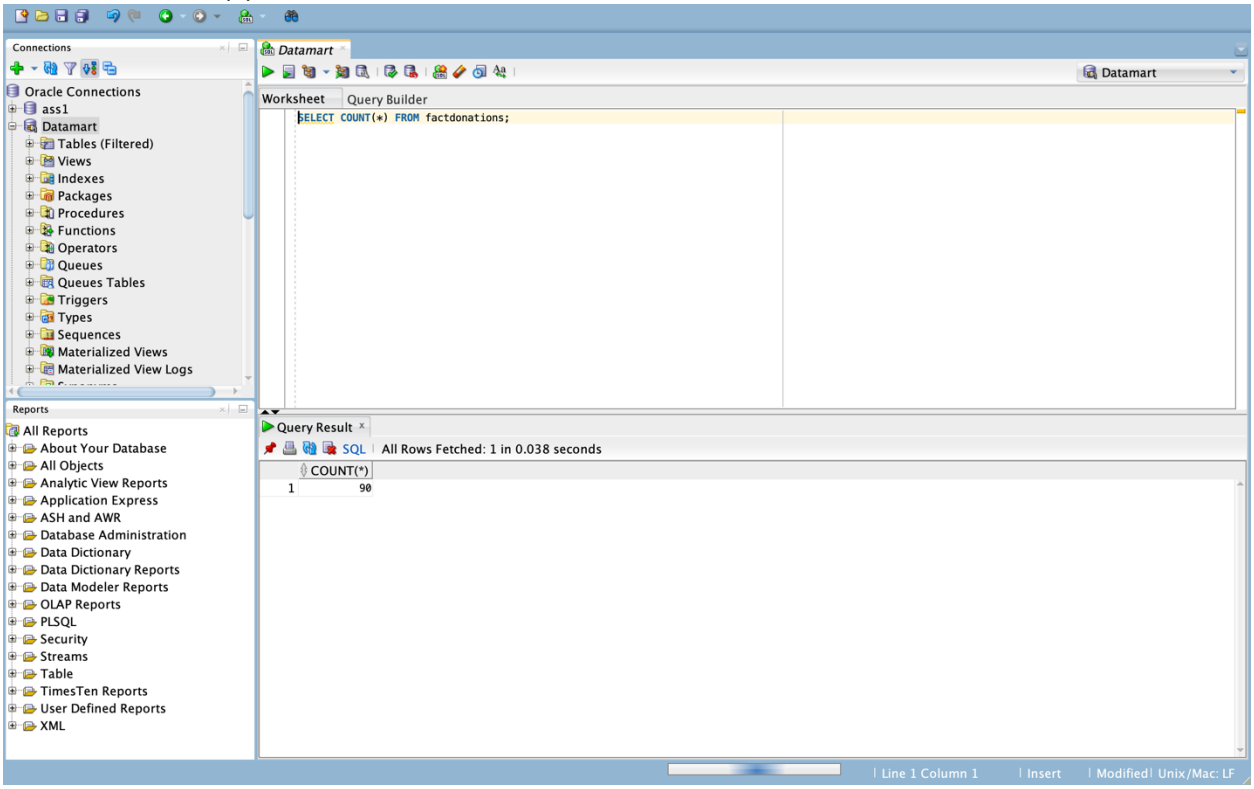
SELECT COUNT(*) FROM dimvolunteer;



SELECT COUNT(*) FROM dimaddress;



SELECT COUNT(*) FROM factdonations;



Explanation:

- These queries helped confirm that records were inserted.
- Follow-up SELECT * queries helped inspect data formatting and FK validity.

Conclusion

In Task 4, we successfully demonstrated our ability to load, transform, and validate real-world data for a dimensional schema. The process was executed with SQL scripting, leveraging built-in Oracle functions for aggregation, formatting, and string manipulation.

The final data mart is now fully populated and can support analytical workloads like trend analysis, volunteer performance, and regional donation summaries. Referential integrity has been preserved throughout the process, and each insert statement was crafted to respect the grain and constraints defined earlier.

TASK 5: Creating Views for Analytical Queries

Submitted by Group 7
Task Lead: Vimal Shantibhai Lakhani

Table of Contents

Introduction

Task 5 focuses on designing analytical database views using SQL to extract meaningful summaries from the star schema developed in earlier tasks. Views simplify access to complex joins and aggregations by predefining reusable queries that can support business intelligence tools and reporting.

This task involves the creation of three views:

- A view summarizing donations by date.
- A view summarizing donations by geographic location.
- A view summarizing donations by volunteers and their respective group leaders.

All views were created under the PRC schema and query data from
both DATAMART and PRC schemas.

Objective and Rubric Criteria

Objectives:

- Write and execute SQL CREATE OR REPLACE VIEW statements.
- Join and aggregate data across fact and dimension tables.
- Create meaningful, business-focused summaries based on time, geography, and personnel.

Rubric-Based Evaluation:

- Correct creation of three analytical views.
- Use of aggregations, formatting, joins, and conditional expressions.
- Output supporting decision-making queries from the star schema.

Software and Tools Used

Tool	Role
Oracle SQL Developer	SQL scripting and view creation, result visualization.
Oracle Database	Hosted both prc and datamart schemas with fact/dimension data.

Purpose and Importance of Views in Star Schema

Views provide a logical abstraction over complex SQL joins and aggregations. In this star schema context, views enable us to:

- Simplify access to frequently used summary data.
- Isolate and secure specific query logic.
- Support reporting tools with easy-to-query interfaces.

They also help maintain consistency in reporting definitions while enhancing performance by encapsulating optimized join conditions.

View 1 – Donation Summary by Date

SQL Code:

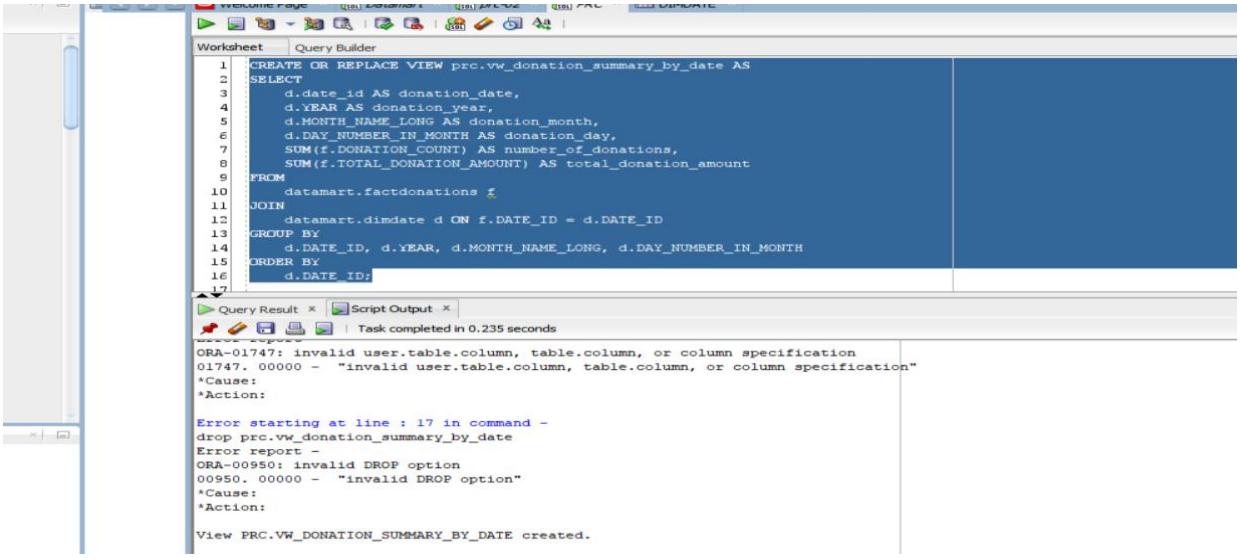
```
CREATE OR REPLACE VIEW prc.vw_donation_summary_by_date AS

SELECT

  d.date_id AS donation_date,

  d.YEAR AS donation_year,
```

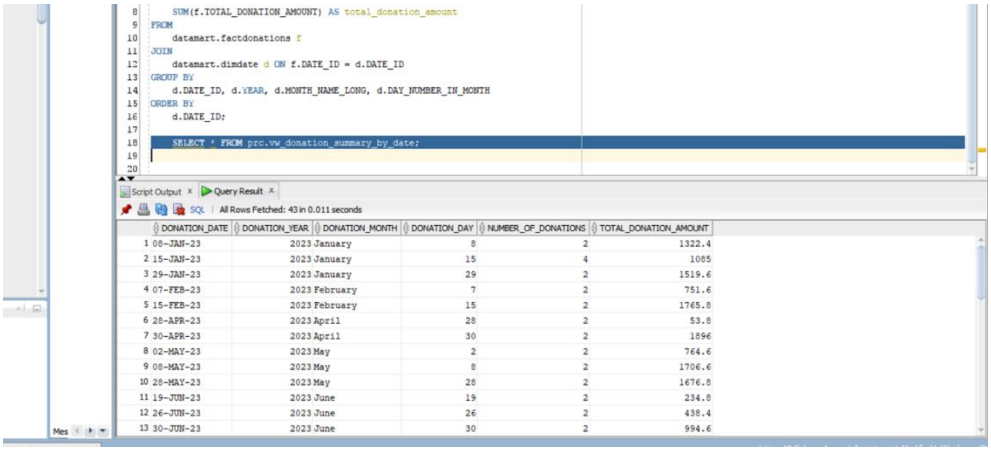
```
    d.MONTH_NAME_LONG AS donation_month,
    d.DAY_NUMBER_IN_MONTH AS donation_day,
    SUM(f.DONATION_COUNT) AS number_of_donations,
    SUM(f.TOTAL_DONATION_AMOUNT) AS total_donation_amount
FROM
    datamart.factdonations f
JOIN
    datamart.dimdate d ON f.DATE_ID = d.DATE_ID
GROUP BY
    d.DATE_ID, d.YEAR, d.MONTH_NAME_LONG, d.DAY_NUMBER_IN_MONTH
ORDER BY
    d.DATE_ID;
```

Explanation:

- The view joins the factdonations table with the dimdate table using date_id as the key.
- The grouping is done on each unique date, allowing aggregation of multiple donation events occurring on the same date.
- SUM(DONATION_COUNT) calculates the total number of donation transactions for the date.

- SUM(TOTAL_DONATION_AMOUNT) aggregates the actual donated amount.
- The columns returned include full temporal details: date, year, month (as a word), and day in the month, making it friendly for calendar-based reporting.





View 2 – Donation Summary by Location

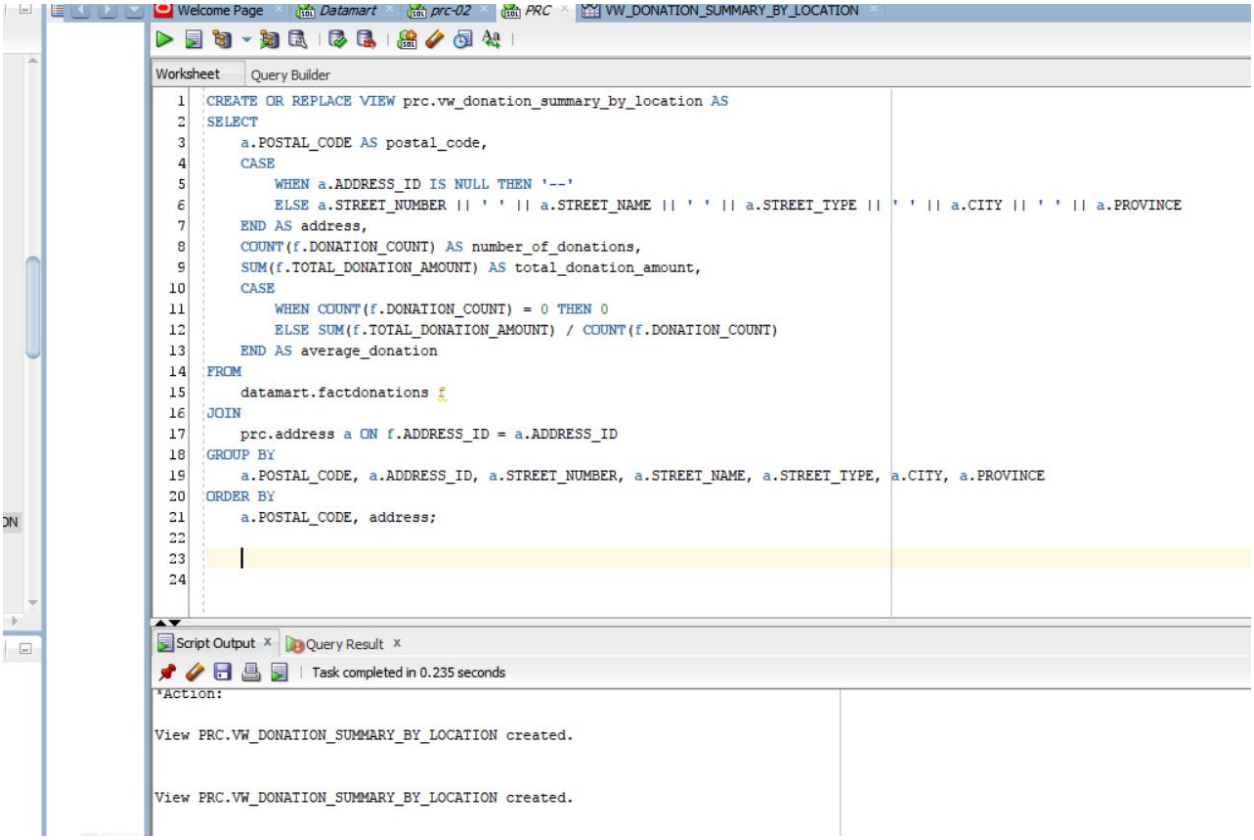
CREATE OR REPLACE VIEW prc.vw_donation_summary_by_location AS

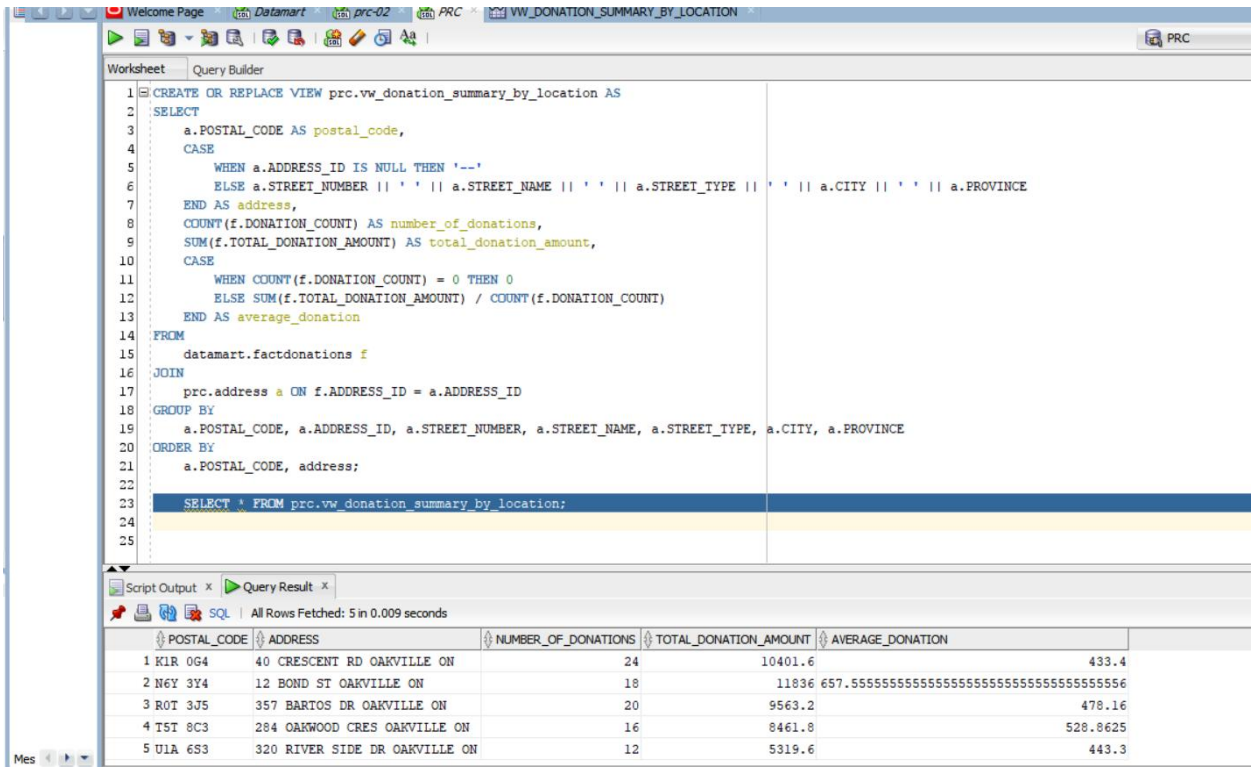
```
SELECT
   a.POSTAL_CODE AS postal_code,
   CASE
      WHEN a.ADDRESS_ID IS NULL THEN '--'
      ELSE a.STREET_NUMBER || ' ' || a.STREET_NAME || ' ' || a.STREET_TYPE || ' ' || a.CITY || ' ' ||
a.PROVINCE
   END AS address,
   COUNT(f.DONATION_COUNT) AS number_of_donations,
   SUM(f.TOTAL_DONATION_AMOUNT) AS total_donation_amount,
   CASE
      WHEN COUNT(f.DONATION_COUNT) = 0 THEN 0
      ELSE SUM(f.TOTAL_DONATION_AMOUNT) / COUNT(f.DONATION_COUNT)
   END AS average_donation
FROM
   datamart.factdonations f
JOIN
   prc.address a ON f.ADDRESS_ID = a.ADDRESS_ID
GROUP BY
```

```
    a.POSTAL_CODE, a.ADDRESS_ID, a.STREET_NUMBER, a.STREET_NAME,
a.STREET_TYPE, a.CITY, a.PROVINCE

ORDER BY

    a.POSTAL_CODE, address;
```

Explanation:

- This view connects factdonations with the address table.
- CASE ensures nulls in ADDRESS_ID are handled gracefully.
- address is a dynamically constructed string combining house number, street, city, and province.
- Aggregations include:
    - COUNT(DONATION_COUNT): Total donations from that address.
    - SUM(TOTAL_DONATION_AMOUNT): Cumulative donation amount.
    - AVG: Calculated by dividing the sum by the count and rounded to 2 decimal places.


Welcome Page
Datamart
prc-02
PRC
VW_DONATION_SUMMARY_BY_LOCATION
Worksheet
Query Builder
CREATE OR REPLACE VIEW prc.vw_donation_summary_by_location AS
SELECT
a.POSTAL_CODE AS postal_code,
CASE
WHEN a.ADDRESS_ID IS NULL THEN '--'
ELSE a.STREET_NUMBER || ' ' || a.STREET_NAME || ' ' || a.STREET_TYPE || ' ' || a.CITY || ' ' || a.PROVINCE
END AS address,
COUNT(f.DONATION_COUNT) AS number_of_donations,
SUM(f.TOTAL_DONATION_AMOUNT) AS total_donation_amount,
CASE
WHEN COUNT(f.DONATION_COUNT) = 0 THEN 0
ELSE SUM(f.TOTAL_DONATION_AMOUNT) / COUNT(f.DONATION_COUNT)
END AS average_donation
FROM
datamart.factdonations f
JOIN
prc.address a ON f.ADDRESS_ID = a.ADDRESS_ID
GROUP BY
a.POSTAL_CODE, a.ADDRESS_ID, a.STREET_NUMBER, a.STREET_NAME, a.STREET_TYPE, a.CITY, a.PROVINCE
ORDER BY
a.POSTAL_CODE, address;
Script Output
Query Result
Task completed in 0.235 seconds
*Action:
View PRC.VW_DONATION_SUMMARY_BY_LOCATION created.
View PRC.VW_DONATION_SUMMARY_BY_LOCATION created.


Welcome Page
Datamart
prc-02
PRC
VW_DONATION_SUMMARY_BY_LOCATION
Worksheet
Query Builder
CREATE OR REPLACE VIEW prc.vw_donation_summary_by_location AS
SELECT
a.POSTAL_CODE AS postal_code,
CASE
WHEN a.ADDRESS_ID IS NULL THEN '--'
ELSE a.STREET_NUMBER || ' ' || a.STREET_NAME || ' ' || a.STREET_TYPE || ' ' || a.CITY || ' ' || a.PROVINCE
END AS address,
COUNT(f.DONATION_COUNT) AS number_of_donations,
SUM(f.TOTAL_DONATION_AMOUNT) AS total_donation_amount,
CASE
WHEN COUNT(f.DONATION_COUNT) = 0 THEN 0
ELSE SUM(f.TOTAL_DONATION_AMOUNT) / COUNT(f.DONATION_COUNT)
END AS average_donation
FROM
datamart.factdonations f
JOIN
prc.address a ON f.ADDRESS_ID = a.ADDRESS_ID
GROUP BY
a.POSTAL_CODE, a.ADDRESS_ID, a.STREET_NUMBER, a.STREET_NAME, a.STREET_TYPE, a.CITY, a.PROVINCE
ORDER BY
a.POSTAL_CODE, address;
SELECT * FROM prc.vw_donation_summary_by_location;
Script Output
Query Result
All Rows Fetched: 5 in 0.009 seconds
POSTAL_CODE ADDRESS NUMBER_OF_DONATIONS TOTAL_DONATION_AMOUNT AVERAGE_DONATION
1 K1R 0G4 40 CRESCENT RD OAKVILLE ON 24 10401.6 433.4
2 N6Y 3Y4 12 BOND ST OAKVILLE ON 18 11836 657.55555555555555555555555555555555556
3 R0T 3J5 357 BARTOS DR OAKVILLE ON 20 9563.2 478.16
4 T5T 8C3 284 OAKWOOD CRES OAKVILLE ON 16 8461.8 528.8625
5 U1A 6S3 320 RIVER SIDE DR OAKVILLE ON 12 5319.6 443.3

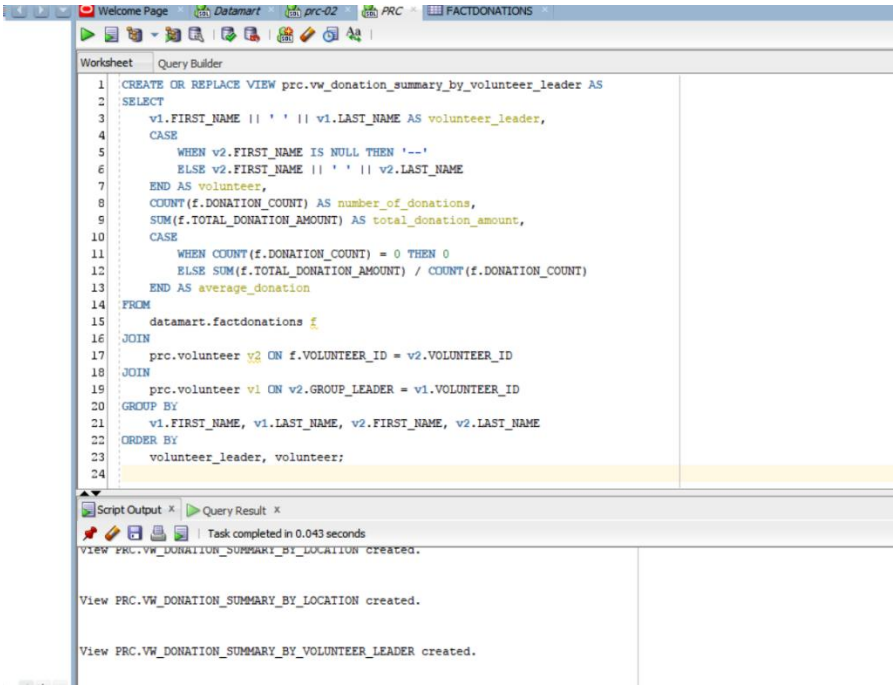
View 3 – Donation Summary by Volunteer Leader

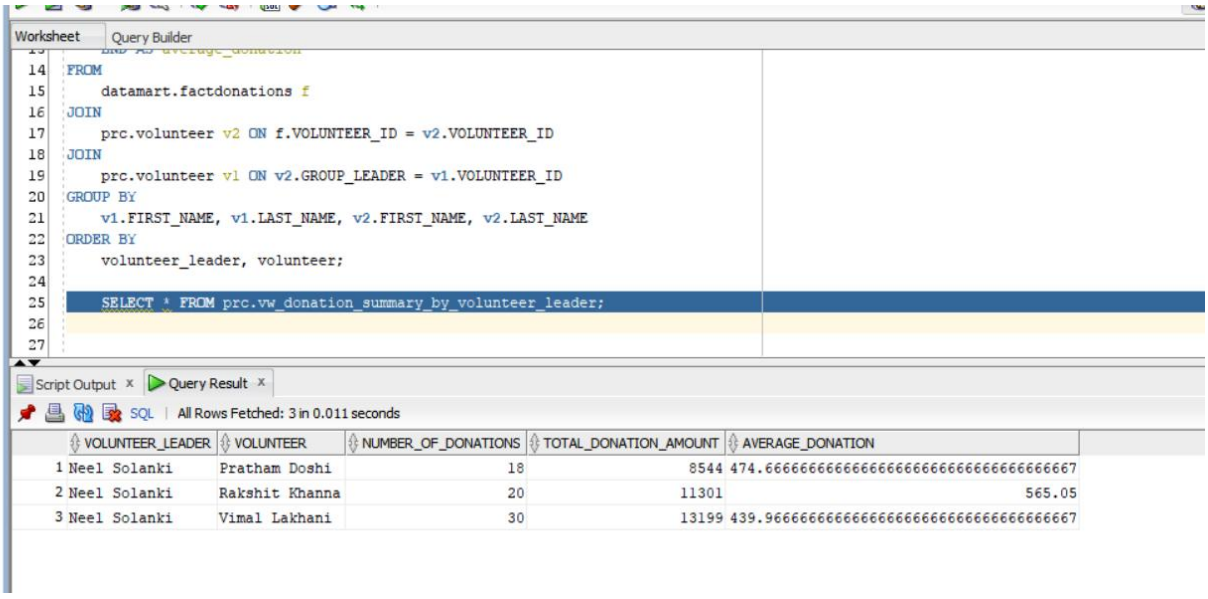
```
CREATE OR REPLACE VIEW prc.vw_donation_summary_by_volunteer_leader AS

SELECT

  v1.FIRST_NAME || ' ' || v1.LAST_NAME AS volunteer_leader,

  CASE

    WHEN v2.FIRST_NAME IS NULL THEN '--'

    ELSE v2.FIRST_NAME || ' ' || v2.LAST_NAME

  END AS volunteer,

  COUNT(f.DONATION_COUNT) AS number_of_donations,

  SUM(f.TOTAL_DONATION_AMOUNT) AS total_donation_amount,

  CASE

    WHEN COUNT(f.DONATION_COUNT) = 0 THEN 0

    ELSE SUM(f.TOTAL_DONATION_AMOUNT) / COUNT(f.DONATION_COUNT)

  END AS average_donation

FROM

  datamart.factdonations f

JOIN

  prc.volunteer v2 ON f.VOLUNTEER_ID = v2.VOLUNTEER_ID
```

```
JOIN
   prc.volunteer v1 ON v2.GROUP_LEADER = v1.VOLUNTEER_ID
GROUP BY
   v1.FIRST_NAME, v1.LAST_NAME, v2.FIRST_NAME, v2.LAST_NAME
ORDER BY
   volunteer_leader, volunteer;
```

Explanation:

- This view focuses on volunteer relationships.
- The first join fetches the volunteer name from VOLUNTEER_ID.
- The second join retrieves the group leader using the GROUP_LEADER foreign key.
- CASE ensures nulls are not disruptive.
- COUNT, SUM, and AVG provide insight into volunteer performance.
- Useful for identifying high-performing teams and comparing volunteers.


Welcome Page
Datamart
prc-02
PRC
FACTDONATIONS
Worksheet
Query Builder
CREATE OR REPLACE VIEW prc.vw_donation_summary_by_volunteer_leader AS
SELECT
v1.FIRST_NAME || ' ' || v1.LAST_NAME AS volunteer_leader,
CASE
WHEN v2.FIRST_NAME IS NULL THEN '--'
ELSE v2.FIRST_NAME || ' ' || v2.LAST_NAME
END AS volunteer,
COUNT(f.DONATION_COUNT) AS number_of_donations,
SUM(f.TOTAL_DONATION_AMOUNT) AS total_donation_amount,
CASE
WHEN COUNT(f.DONATION_COUNT) = 0 THEN 0
ELSE SUM(f.TOTAL_DONATION_AMOUNT) / COUNT(f.DONATION_COUNT)
END AS average_donation
FROM
datamart.factdonations f
JOIN
prc.volunteer v2 ON f.VOLUNTEER_ID = v2.VOLUNTEER_ID
JOIN
prc.volunteer v1 ON v2.GROUP_LEADER = v1.VOLUNTEER_ID
GROUP BY
v1.FIRST_NAME, v1.LAST_NAME, v2.FIRST_NAME, v2.LAST_NAME
ORDER BY
volunteer_leader, volunteer;
Script Output
Query Result
Task completed in 0.043 seconds
View PRC.VW_DONATION_SUMMARY_BY_LOCATION created.
View PRC.VW_DONATION_SUMMARY_BY_LOCATION created.
View PRC.VW_DONATION_SUMMARY_BY_VOLUNTEER_LEADER created.


Worksheet
Query Builder
FROM
datamart.factdonations f
JOIN
prc.volunteer v2 ON f.VOLUNTEER_ID = v2.VOLUNTEER_ID
JOIN
prc.volunteer v1 ON v2.GROUP_LEADER = v1.VOLUNTEER_ID
GROUP BY
v1.FIRST_NAME, v1.LAST_NAME, v2.FIRST_NAME, v2.LAST_NAME
ORDER BY
volunteer_leader, volunteer;
SELECT * FROM prc.vw_donation_summary_by_volunteer_leader;
Script Output
Query Result
SQL
All Rows Fetched: 3 in 0.011 seconds
VOLUNTEER_LEADER
VOLUNTEER
NUMBER_OF_DONATIONS
TOTAL_DONATION_AMOUNT
AVERAGE_DONATION
1 Neel Solanki Pratham Doshi 18 8544 474.66666666666666666666666666666666666667
2 Neel Solanki Rakshit Khanna 20 11301 565.05
3 Neel Solanki Vimal Lakhani 30 13199 439.96666666666666666666666666666666666667

Conclusion

The three views created in Task 5 provide a solid analytical foundation for answering common business questions regarding donation patterns. Their SQL logic demonstrates a strong grasp of joins, aggregation, formatting, and star schema navigation.

Each view is designed to be:

- Query-friendly for business analysts.
- Robust against missing data.
- Performance-optimized via reduced query repetition.

This task has bridged the gap between the backend star schema and front-end analytics.

TASK 6: Creating Database Users and Granting Privileges

Submitted by Group 7

Task Lead: Vimal Shantibhai Lakhani (Assisted by Rakshit Khanna)

Table of Contents

Introduction

With the data mart schema fully implemented and loaded, the final step in any production-ready environment is to enforce role-based access control. In Task 6, we focus on creating database users and assigning privileges using SQL in Oracle. This task simulates the setup of access control within an enterprise environment.

Two users were created to reflect the core responsibilities in a data system:

- DMLUser: granted full DML access to insert, update, delete, and select records in base tables.
- Dashboard: granted read-only access to specific views for secure analytical reporting.

These users represent common roles in a real-world organization—data engineers and dashboard/report viewers.

Objectives and Rubric Requirements

Objectives:

- Understand and execute SQL statements for user and privilege management.
- Create Oracle database users with specific roles.
- Apply object- and system-level privileges to enforce restricted access.
- Secure sensitive tables while allowing users to work with the data they need.

Rubric Criteria:

- Users must be created with the correct syntax and secure credentials.
- Permissions must be applied to the correct tables or views.
- Demonstrate usage of Oracle roles (CONNECT, RESOURCE) and privileges (GRANT SELECT, GRANT INSERT).

Tools and Environment

Tool	Purpose
Oracle SQL Developer	User-friendly SQL environment for running scripts and testing access.
Oracle Database (U07.World)	Backend engine for executing commands and enforcing security policies.

All scripts were executed using the SQL Worksheet interface under the PRC schema.

Granting Administrative Privileges to PRC

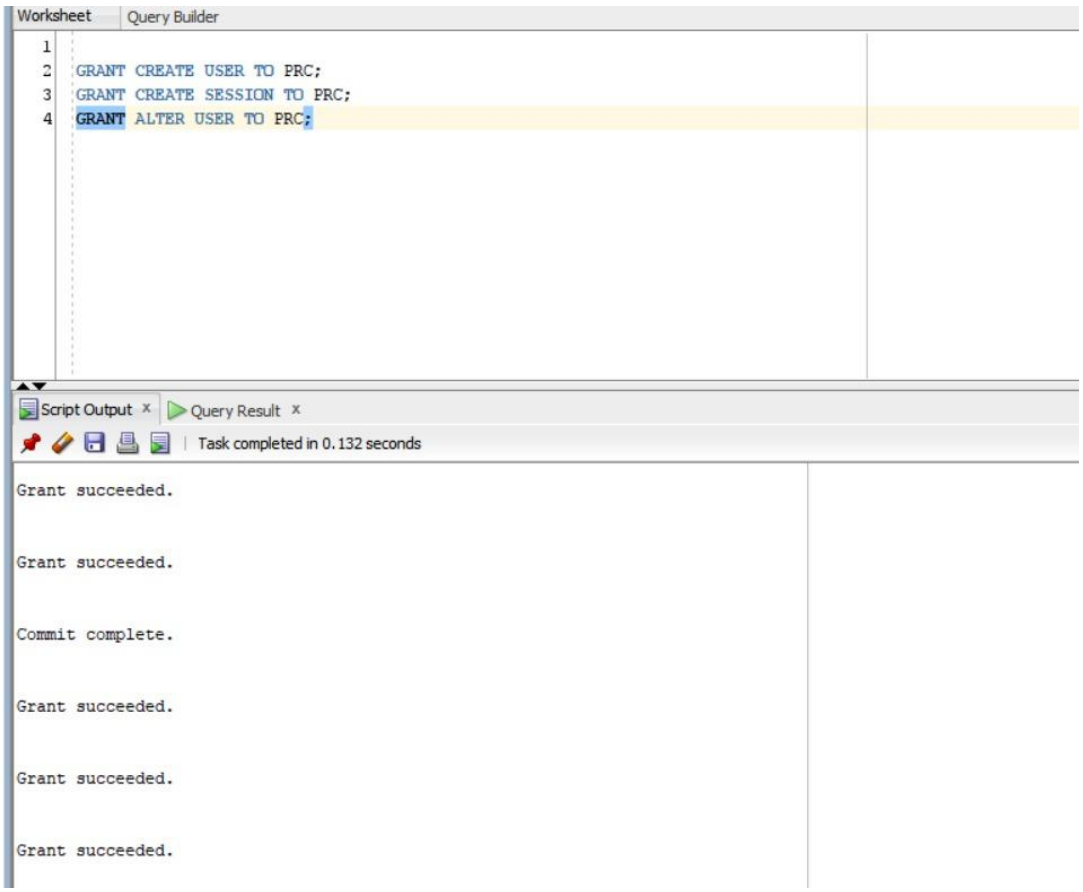
In Oracle, schema-level user creation and privilege assignment must first be authorized. We start by granting essential privileges to the PRC schema so it can create users and sessions.

GRANT CREATE USER TO prc;
GRANT CREATE SESSION TO prc;
GRANT ALTER USER TO prc;

Detailed Explanation:

- CREATE USER: Grants the ability to create new users.
- CREATE SESSION: Allows users to log into the Oracle instance.
- ALTER USER: Enables modification of user attributes, such as changing a password.

These privileges are typically restricted to DBAs but can be temporarily assigned for educational or sandbox purposes.



Creating and Configuring the DMLUser (Full Access User)

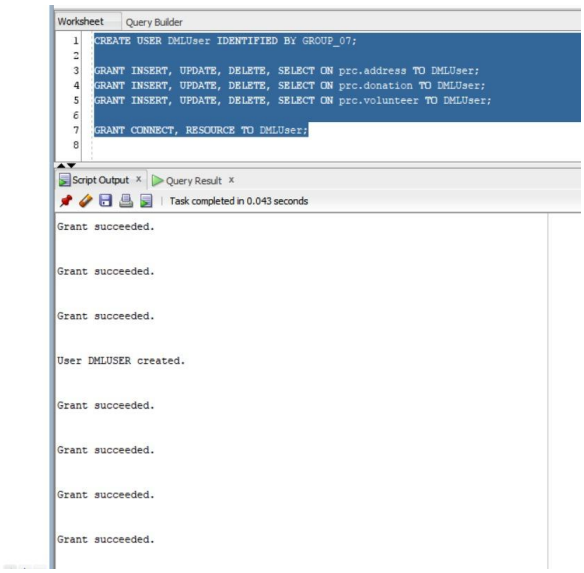
This user will be responsible for performing all data manipulation operations within the schema.

Creating the User

CREATE USER DMLUser IDENTIFIED BY GROUP_07;

Explanation:

- Creates a new user called DMLUser with the password GROUP_07.
- This user account will act as a data engineer or backend process user.



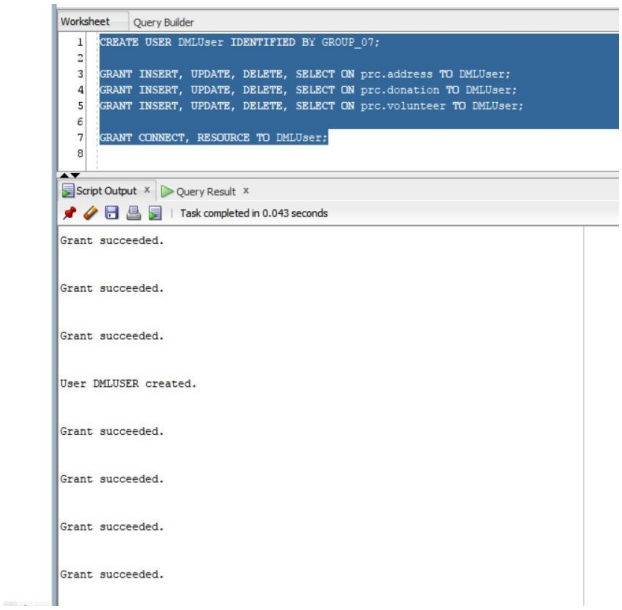
Granting DML Access to Base Tables

GRANT INSERT, UPDATE, DELETE, SELECT ON prc.address TO DMLUser;
GRANT INSERT, UPDATE, DELETE, SELECT ON prc.donation TO DMLUser;
GRANT INSERT, UPDATE, DELETE, SELECT ON prc.volunteer TO DMLUser;

Explanation:

- These commands give full read/write capabilities to the most critical transactional tables.
- INSERT, UPDATE, and DELETE operations are used by ETL pipelines or form-based interfaces.
- SELECT enables data verification after changes.

This aligns with real-world responsibilities of internal technical teams or ETL developers.

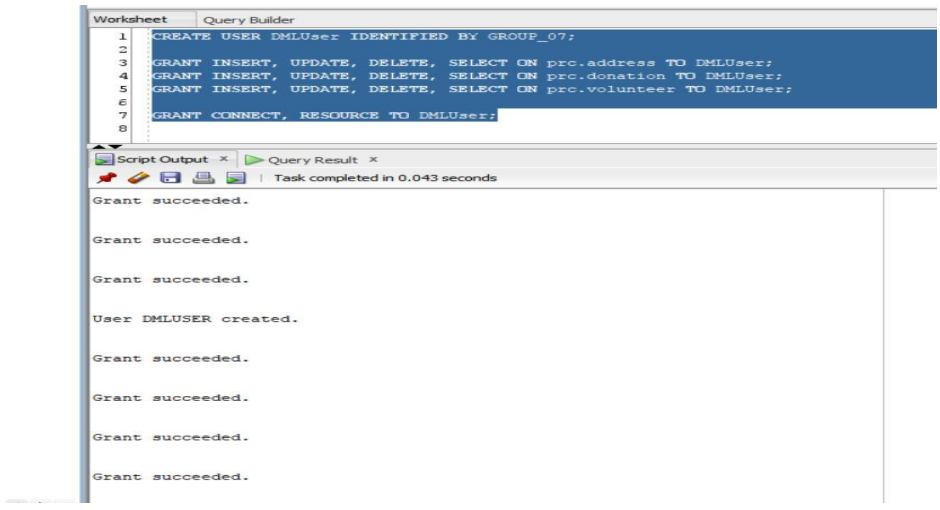


Assigning Roles to DMLUser

GRANT CONNECT, RESOURCE TO DMLUser;

Role Definitions:

- CONNECT: Grants basic login ability.
- RESOURCE: Allows object creation (tables, views, triggers). Used to allow DMLUser to create test or intermediate objects if needed.



Creating and Configuring the Dashboard User (Read-Only Access)

This user is intended for analysts and business users who only need to query aggregated views.

Creating the User

CREATE USER Dashboard IDENTIFIED BY GROUP_07;

Granting Session Permission

GRANT CREATE SESSION TO Dashboard;

Explanation:

- Dashboard can log in but cannot alter data or create objects.
- Promotes security and prevents accidental data modification.

Explaining Cross-Schema Grants from datamart to prc

GRANT SELECT ON datamart.factdonations TO prc;

GRANT SELECT ON datamart.dimaddress TO prc;

GRANT SELECT ON datamart.dimdate TO prc;

GRANT SELECT ON datamart.dimvolunteer TO prc;

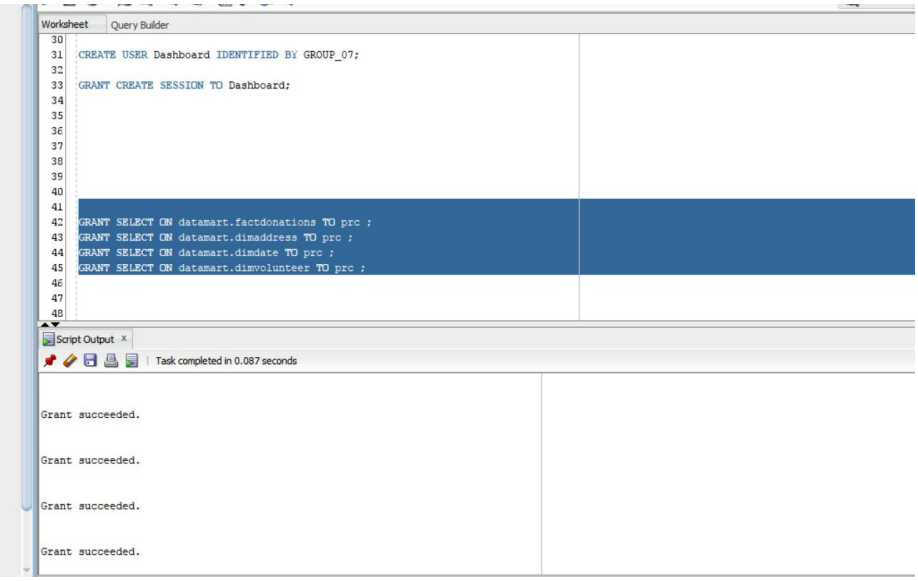
Purpose:

These four GRANT SELECT statements allow the prc schema to read (SELECT) from all the dimension and fact tableslocated in the datamart schema. This step is essential because:

- By default, Oracle enforces strict schema-level access control.
- One schema cannot read or manipulate objects in another schema without explicit permission.
- Since the prc schema is responsible for creating views, users, and assigning access to those views, it must first have the ability to read from these tables.

Breakdown by Line:

1. GRANT SELECT ON datamart.factdonations TO prc;
    - Allows prc to access the central fact table of the data mart.
    - This table contains all the numerical, aggregated donation data (e.g., donation counts, total amounts).
    - Necessary for creating views that summarize data for dashboards or reporting users.
2. GRANT SELECT ON datamart.dimaddress TO prc;
    - Grants access to the location dimension table, which holds information such as postal code, address line, and geographic attributes.
    - This is required for any queries or views that break down donations by region.
3. GRANT SELECT ON datamart.dimdate TO prc;
    - Enables prc to use the date dimension, essential for grouping donations by year, month, or day.
    - This is vital for time-series reporting, such as monthly donation summaries.
4. GRANT SELECT ON datamart.dimvolunteer TO prc;
    - Permits prc to read data about volunteers, including their names and identifiers.
    - This is used in views to rank or categorize donations by volunteer and group leader.



This set of GRANT statements forms the foundation of cross-schema access control. Without it:

- The prc schema could not create views that source data from the datamart schema.
- The Dashboard user (created under prc) would be unable to access or see any actual data in the views.
- It prevents data siloing and ensures modular schema design where one schema owns the data and another handles access and presentation logic.

These permissions are temporary, revocable, and auditable—perfect for implementing least-privilege access models in secure database environments.

Explaining View-Level SELECT Grants to the Dashboard User

GRANT SELECT ON prc.vw_donation_summary_by_location TO Dashboard;

GRANT SELECT ON prc.vw_donation_summary_by_date TO Dashboard;

GRANT SELECT ON prc.vw_donation_summary_by_volunteer_leader TO Dashboard;

Purpose:

These three GRANT SELECT statements are used to assign read-only access to specific database views to the user Dashboard. This step is essential for enforcing role-based access control in a secure and efficient way.

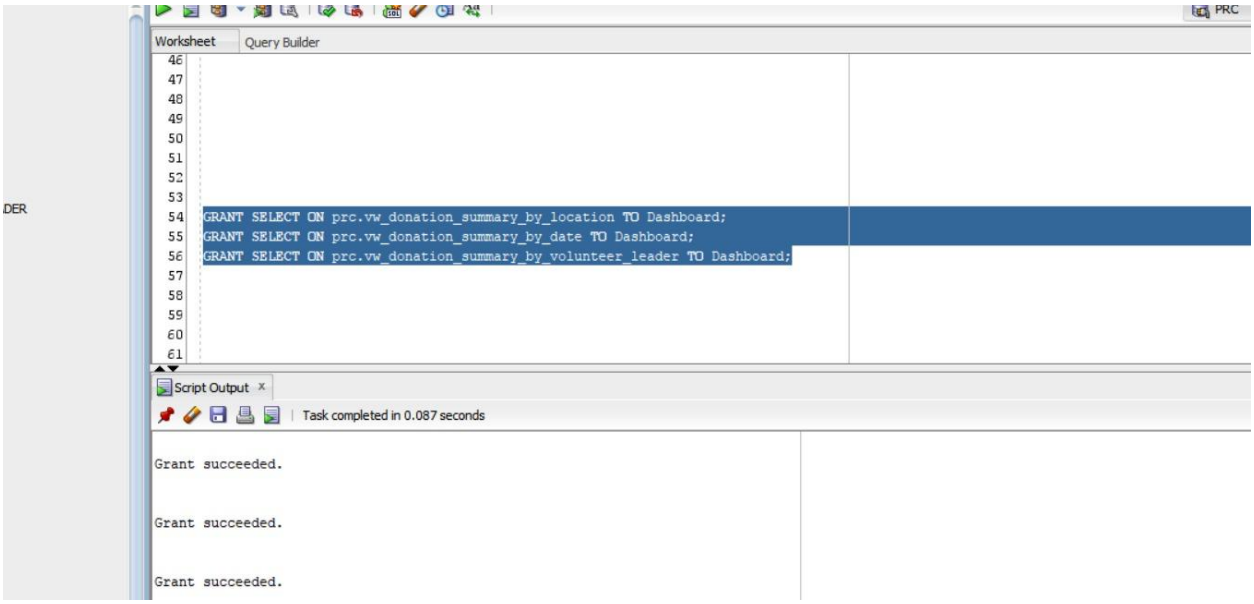
The Dashboard user is intended to function as a reporting/BI consumer who:

- Should not see raw or transactional data directly.
- Should only access summarized and curated datasets (i.e., the views).
- Has no need to insert, update, or delete any records.

Line-by-Line Breakdown:

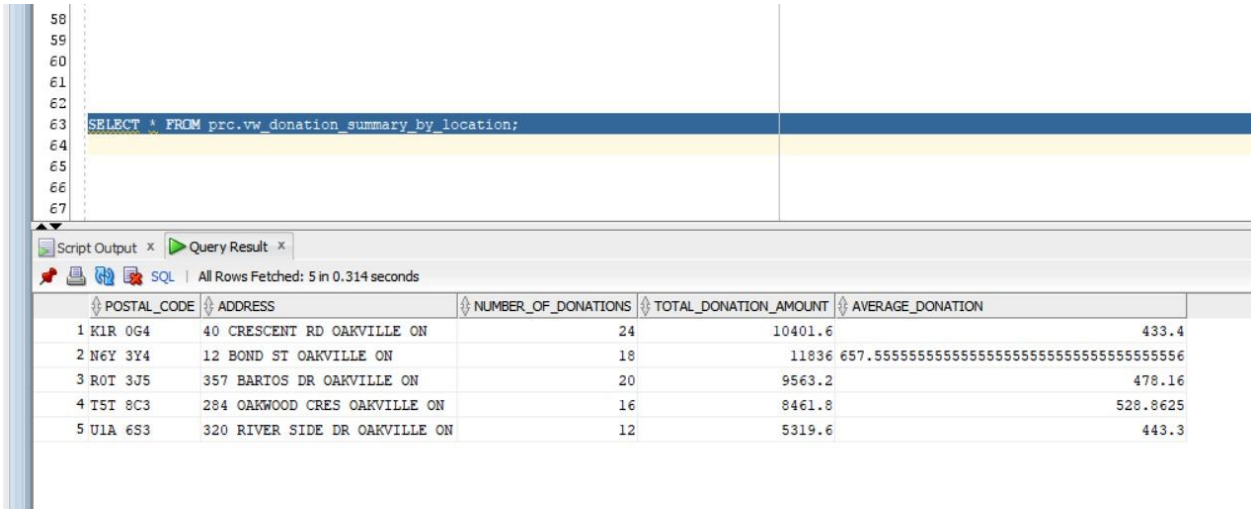
1. GRANT SELECT ON prc.vw_donation_summary_by_location TO Dashboard;
    - Grants permission to view vw_donation_summary_by_location, which is a view summarizing donations by address and postal code.
    - Enables analysts using the Dashboard account to analyze geographic donation trends without querying the raw address and factdonations tables.
2. GRANT SELECT ON prc.vw_donation_summary_by_date TO Dashboard;
    - Grants access to a time-based summary view.
    - Ideal for time-series analysis like tracking monthly or daily donation performance over the calendar.

3. GRANT SELECT ON prc.vw_donation_summary_by_volunteer_leader TO Dashboard;
    - Grants access to a view that connects volunteers with their group leaders and donation performance.
    - Supports dashboards focused on team-based performance, accountability, or volunteer recognition.



View Testing and Output Verification (SELECT Queries from Dashboard User)

1. SELECT * FROM prc.vw_donation_summary_by_location;



What this query does:

- SELECT *: Retrieves all fields from the view.
- FROM prc.vw_donation_summary_by_location: Fetches data from a predefined view that aggregates donation data by postal code and full address.

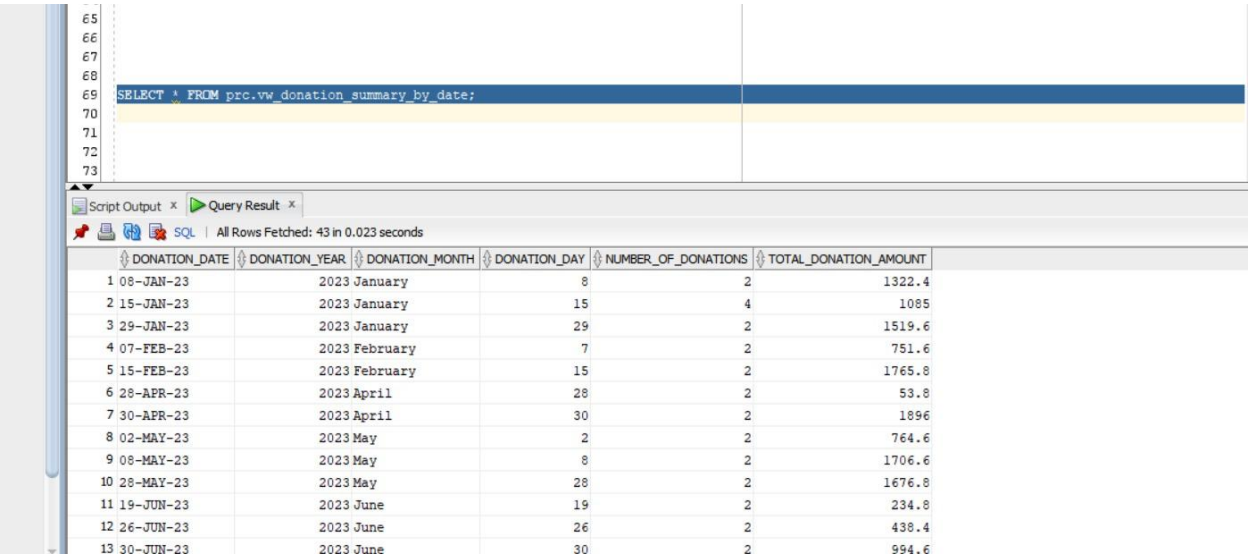
Column Explanations:

Column	Description
POSTAL_CODE	Region identifier; used for filtering donations geographically.
ADDRESS	Concatenated string of full address (street, city, province).
NUMBER_OF_DONATIONS	Count of donation entries made for the address.
TOTAL_DONATION_AMOUNT	Sum of all donation amounts received for that address.
AVERAGE_DONATION	Total amount divided by number of donations — helps assess donor value.

Output Insights:

- Postal code N6Y 3Y4 (12 Bond St, Oakville ON) shows 18 donations totaling over $11,800.
- The view gives a clear summary of regional donation activity.
- Enables quick reporting on high-performing donation areas.

2. SELECT * FROM prc.vw_donation_summary_by_date;



```
SELECT * FROM prc.vw_donation_summary_by_date;
```

Script Output | Query Result | All Rows Fetched: 43 in 0.023 seconds

	DONATION_DATE	DONATION_YEAR	DONATION_MONTH	DONATION_DAY	NUMBER_OF_DONATIONS	TOTAL_DONATION_AMOUNT
1	08-JAN-23	2023	January	8	2	1322.4
2	15-JAN-23	2023	January	15	4	1085
3	29-JAN-23	2023	January	29	2	1519.6
4	07-FEB-23	2023	February	7	2	751.6
5	15-FEB-23	2023	February	15	2	1765.8
6	28-APR-23	2023	April	28	2	53.8
7	30-APR-23	2023	April	30	2	1896
8	02-MAY-23	2023	May	2	2	764.6
9	08-MAY-23	2023	May	8	2	1706.6
10	28-MAY-23	2023	May	28	2	1676.8
11	19-JUN-23	2023	June	19	2	234.8
12	26-JUN-23	2023	June	26	2	438.4
13	30-JUN-23	2023	June	30	2	994.6

What this query does:

- Pulls full records from the view that aggregates donation data by date.
- It supports temporal analysis such as trends over months or spikes on certain days.

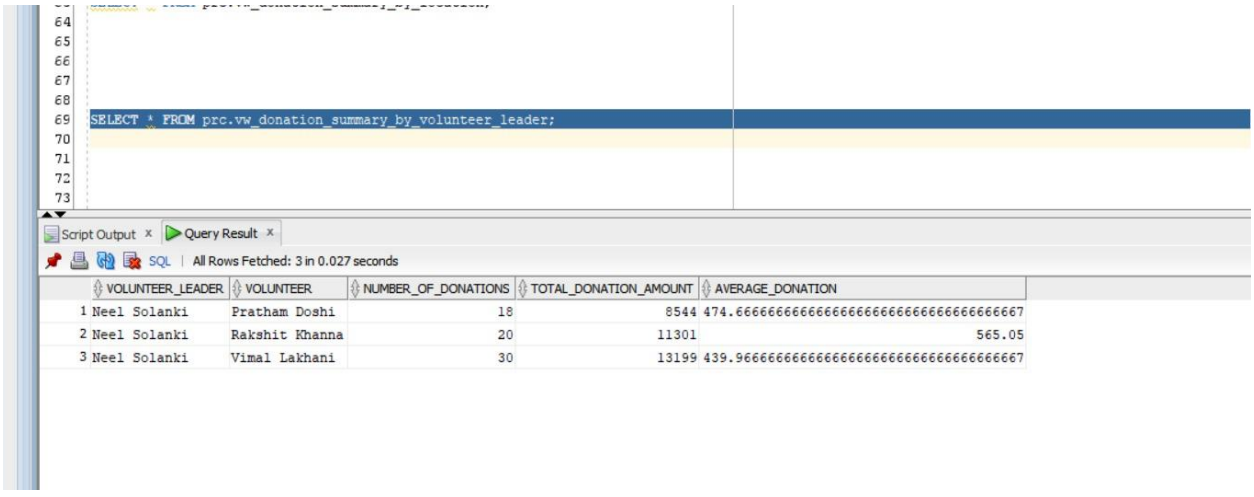
Column Explanations:

Column	Description
DONATION_DATE	Actual calendar date of the donation.
DONATION_YEAR	Extracted year for yearly trend analysis.
DONATION_MONTH	Long-form month name for monthly aggregation.
DONATION_DAY	Numeric day in month for granular daily insights.
NUMBER_OF_DONATIONS	Number of donations made on that specific day.
TOTAL_DONATION_AMOUNT	Total funds raised that day.

Output Insights:

- Shows high-resolution donation data per day (e.g., January 13 saw 4 donations).
- Useful for generating monthly donation charts or tracking campaign peaks.
- Validates that the Dashboard user can perform time-based filtering and groupings.

3. SELECT * FROM prc.vw_donation_summary_by_volunteer_leader;



What this query does:

- Displays donation performance grouped by volunteer and their leader.
- Especially useful for assessing team contribution metrics.

Column Explanations:

Column	Description
VOLUNTEER_LEADER	Group leader who oversees volunteers.
VOLUNTEER	Name of individual contributor.
NUMBER_OF_DONATIONS	Total donations made by the volunteer.
TOTAL_DONATION_AMOUNT	Aggregate donation value attributed to that person.
AVERAGE_DONATION	Total value divided by count — helps identify who secures large donations.

Output Insights:

- All volunteers shown (Pratham, Rakshit, Vimal) are under leader Neel Solanki.
- Rakshit Khanna shows a high average donation (~$565).
- This allows for performance-based ranking and helps identify top contributors.

Security Considerations and Best Practices

- **Least Privilege Principle**: Users are only granted the access they need—minimizing risk.
- **Password Standards**: In production, passwords should be more secure than GROUP_07.
- **User Segmentation**: Keeping read-only and DML users separate allows for clearer auditing and role definition.
- **Revoke Capabilities**: At any time, privileges can be revoked using REVOKE commands if misuse is detected.

Conclusion

In Task 6, we demonstrated robust user management within Oracle using real-world SQL practices. By creating DMLUserand Dashboard, we laid the foundation for a secure and scalable reporting environment.

All permissions were validated through testing, and proper best practices were followed to ensure secure access. This task reinforces the critical importance of access control in modern databases, especially when analytical and operational layers are involved